

多旋翼飞行器设计与控制
系列实验

RflySim高级版 第6讲 视觉控制算法开发

戴训华 daixunhua@163.com
中南大学 计算机学院 副教授
北京航空航天大学 自动化学院 博士



北航可靠飞行控制研究组
BUAA Reliable Flight Control Group

大纲

1. 安装使用方法
2. 基础接口使用
3. 单目视觉控制例子
4. 双目视觉控制例子
5. 小结

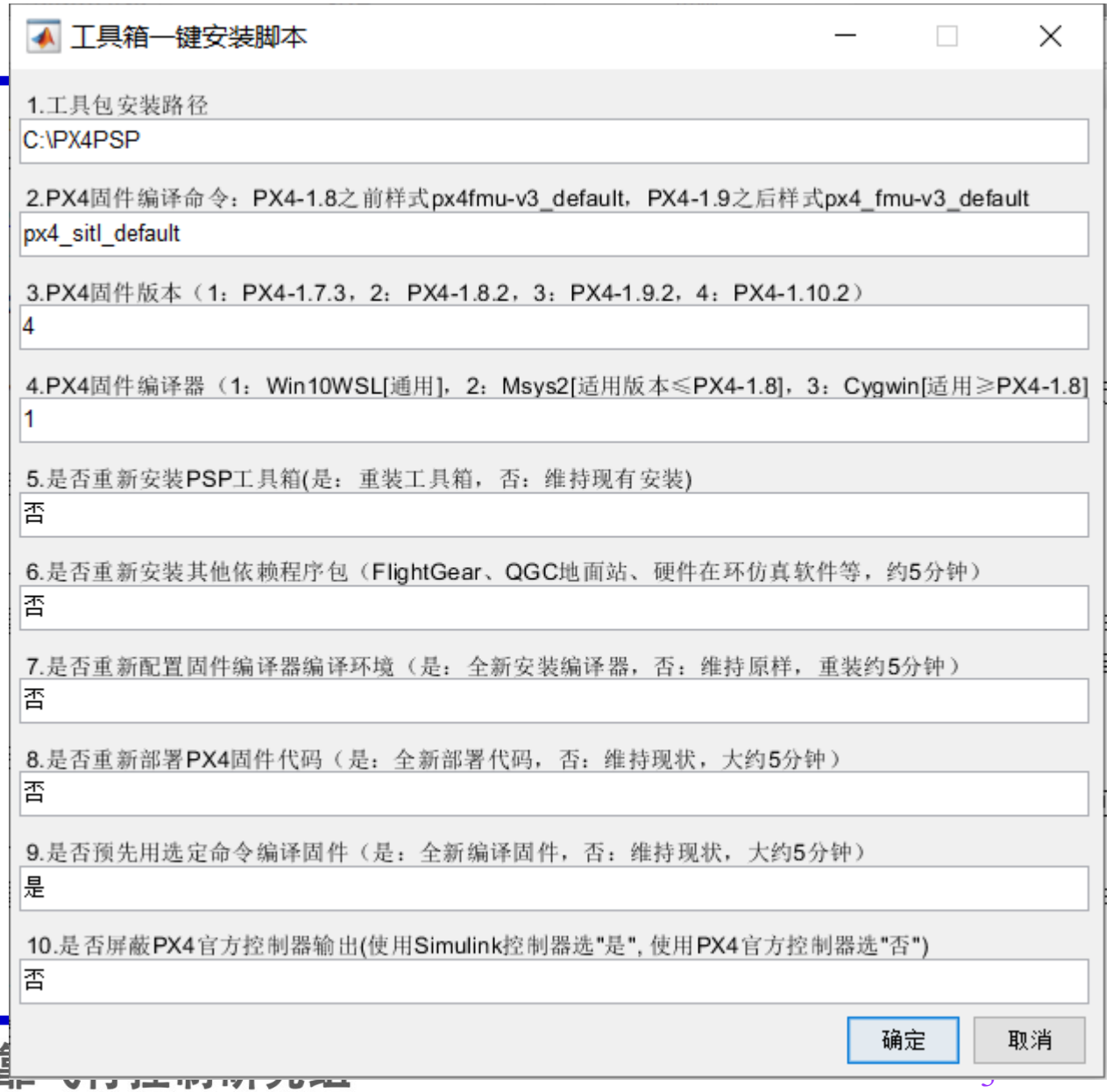
本讲的例程源码路径：
RflySimAPIs\PythonVisionAPI

1.安装使用方法

1.1 RflySim平台配置

- 本平台用于视觉开发时，推荐重新运行“OnekeyScript.p”脚本，并使用右图配置运行脚本如下：

1. 使用PX4 SITL软件在环固件编译命令输入“px4_sitl_default”
2. 使用最新的PX4固件 PX4-1.10.2，固件版本选择“4”
3. 使用Win10WSL编译器，因此编译器选“1”
4. 是否屏蔽PX4输出项目选择“否”
5. 点击“确定”按钮即可开始安装。



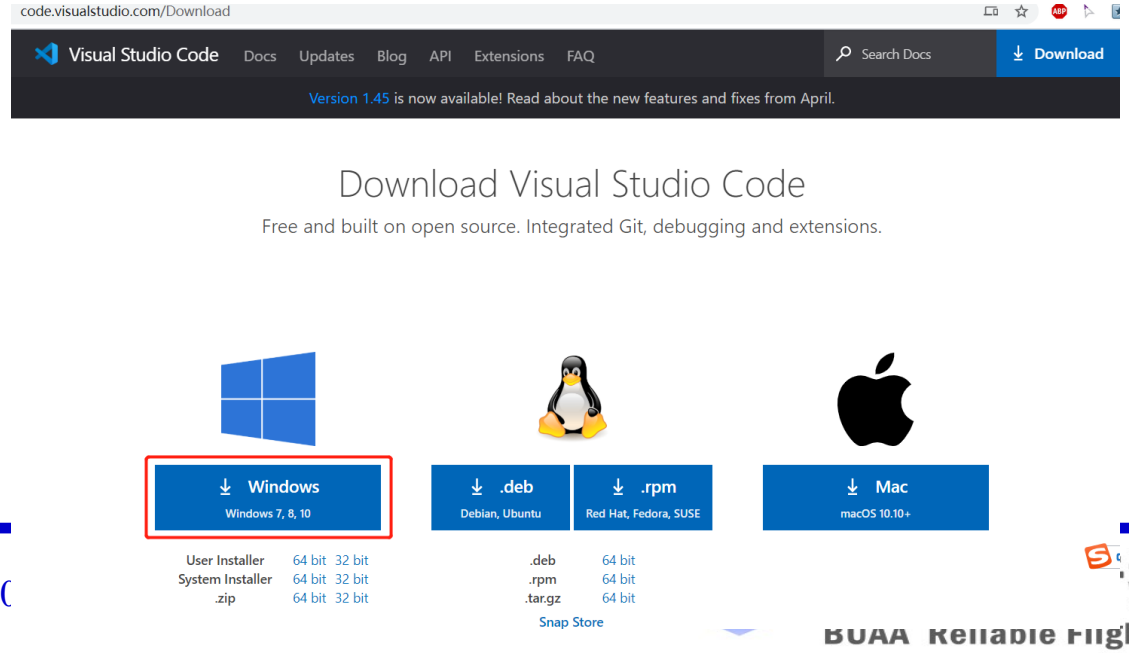
如果要使用Pixhawk进行硬件在环仿真，还需要按照“RflySim高级版_第01讲_总体介绍.pptx”中2.5节方法通过QGC正确配置Pixhawk自驾仪

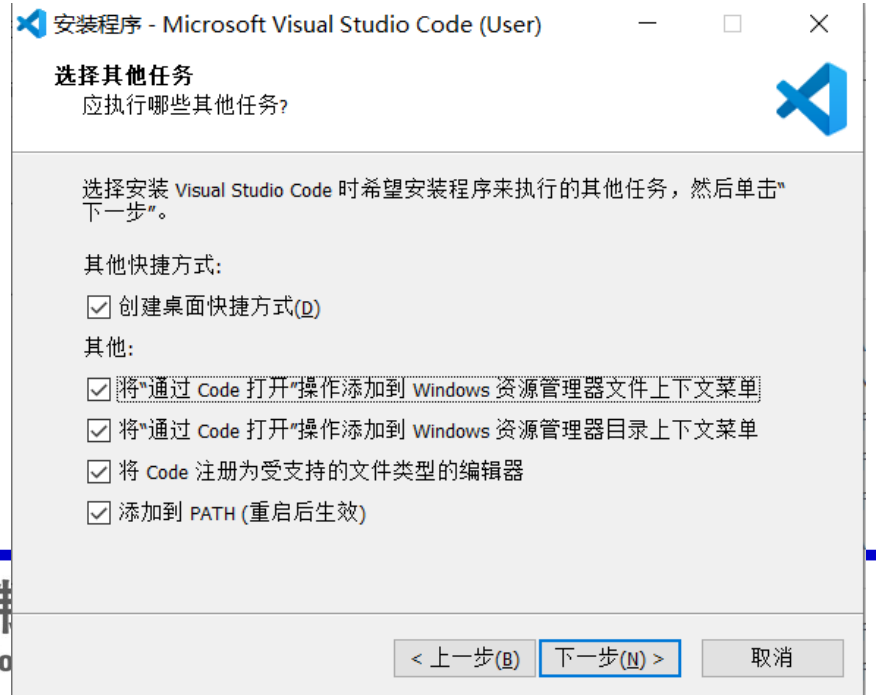
1.安装使用方法

1.2 VS Code编辑器安装

- 本课程的Python编辑器推荐使用VS Code（非必须，但是方便阅读源码和运行），安装步骤如下：

1. 访问 https://code.visualstudio.com/Download 下载最新的VS code安装包（也可使用RflySimAPIs\PythonVisionAPI文件夹下的VSCodeUserSetup-x64-***.exe安装）
2. 安装时，选默认配置即可。需要注意右下图设置，方便直接打开Python文件。

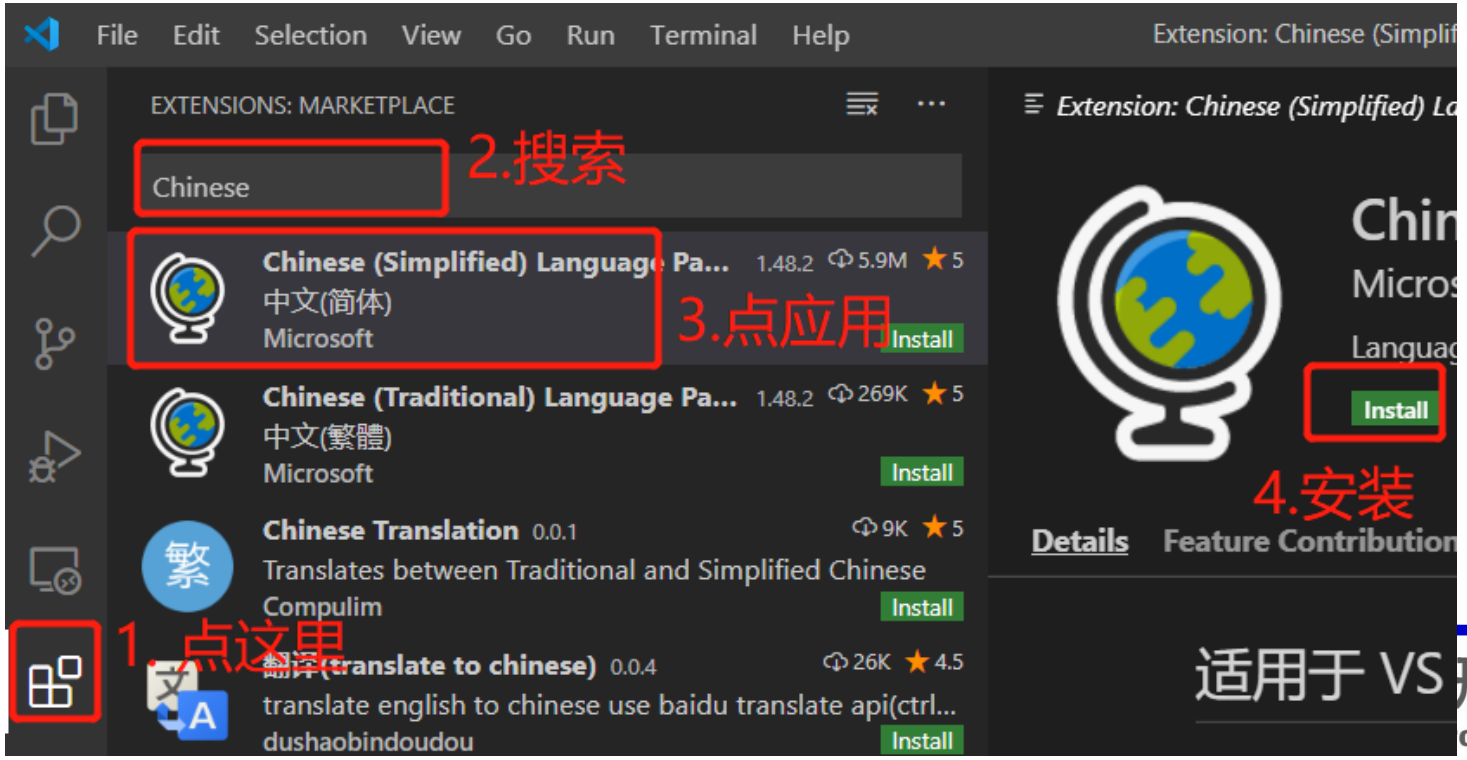


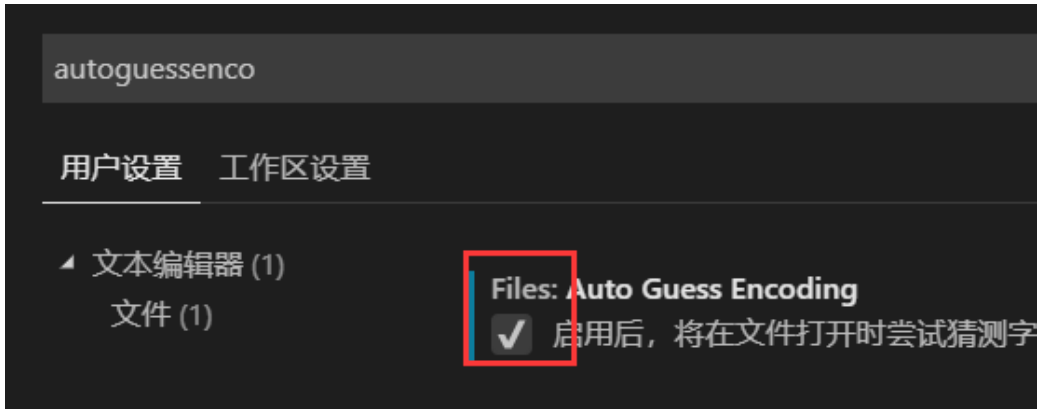


1.安装使用方法

1.3 VS Code编辑器配置

- 打开VS Code，如下图到扩展库界面，依次搜索并安装插件“Chinese (Simplified) Language Pack”和“Python”插件。使得VS Code支持中文和Python。
- 自动识别文件编码（解决中文乱码问题）。菜单栏：文件 → 首选项 → 设置 → 搜索“autoguessencoding”，勾选启用自动猜测文件编码功能

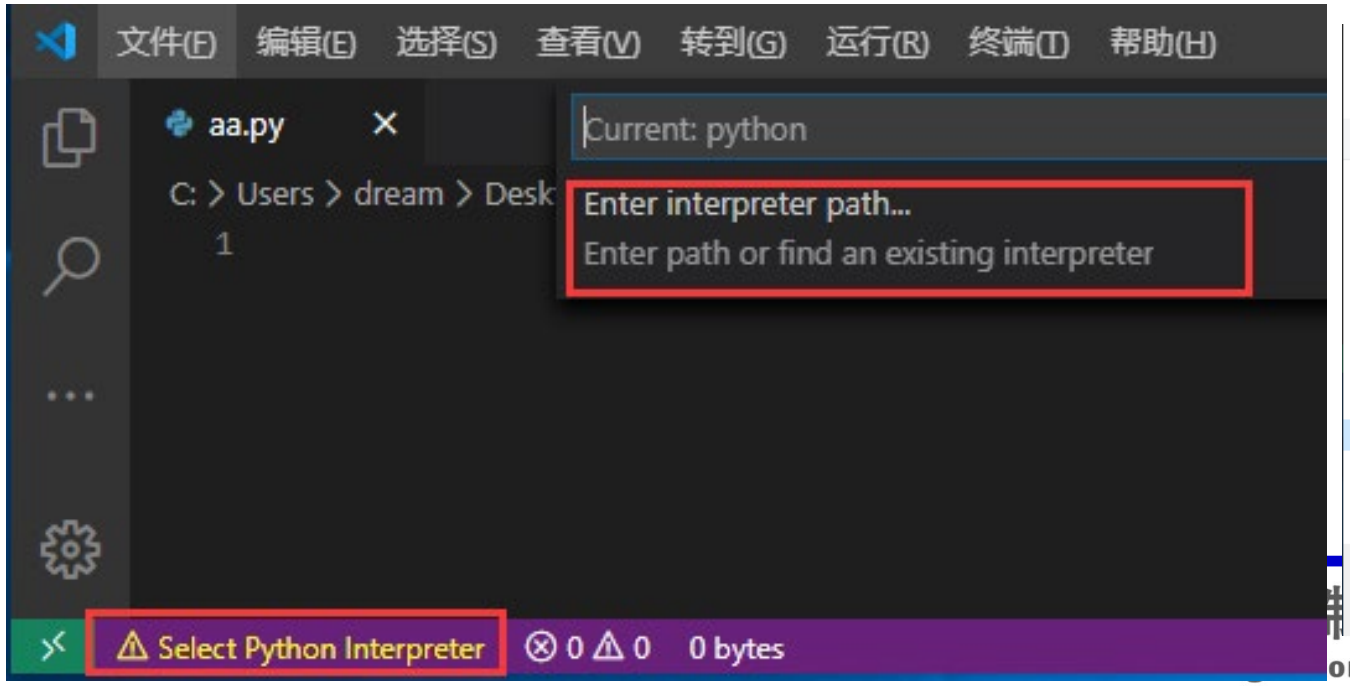


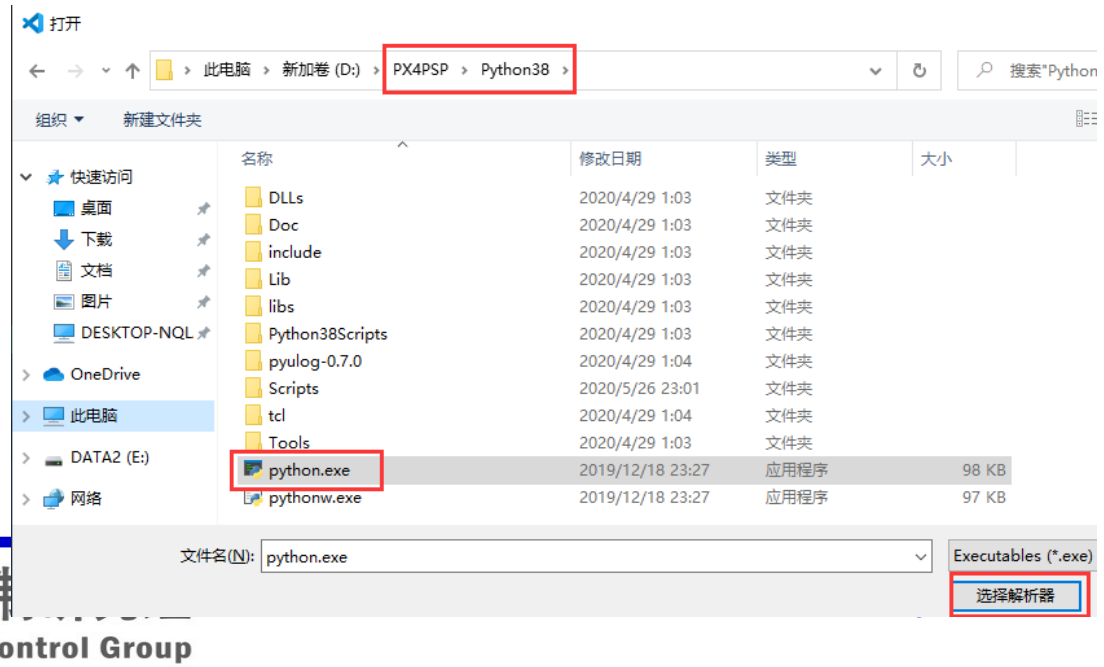


1.安装使用方法

1.4 VS Code Python环境配置

- 用VS code打开“RflySimAPIs\Python38Scripts\ImgCVShow.py”文件（或任意一个.py后缀文件）。如下图所示，点击左下角的黄字“Select Python interpreter”选项，在弹出的项目中点击“Enter interpreter path”。
- 如右下图，在弹出资源浏览窗口中，选择安装目录（默认C:\PX4PSP）下的Python38文件夹内的python.exe文件，点击“选择解析器”。

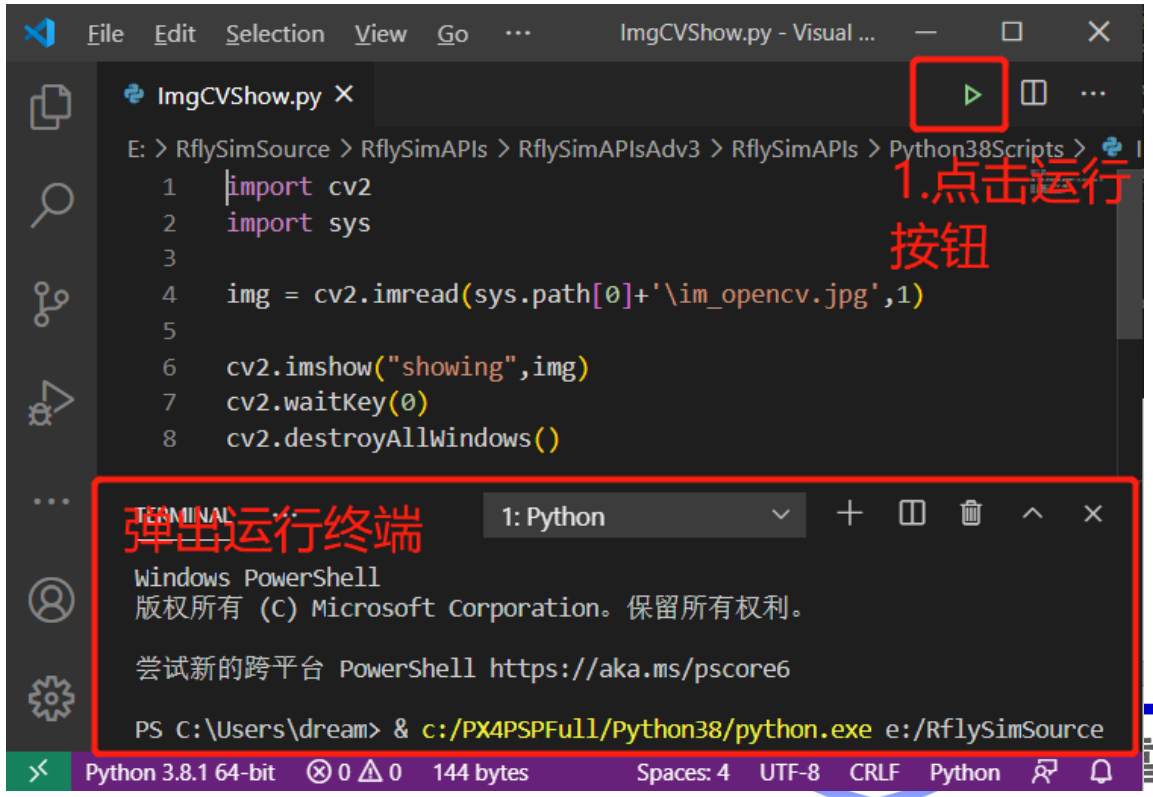




1.安装使用方法

1.4 VS Code Python程序运行

- 用VS Code打开“RflySimAPIs\Python38Scripts\ImgCVShow.py”文件。
- 点击VS Code右上侧的三角箭头即可运行，VS Code界面下方弹出“TERMINAL”终端窗口可查看脚本运行情况，同时弹出右下所示一张图片，说明环境安装正常。

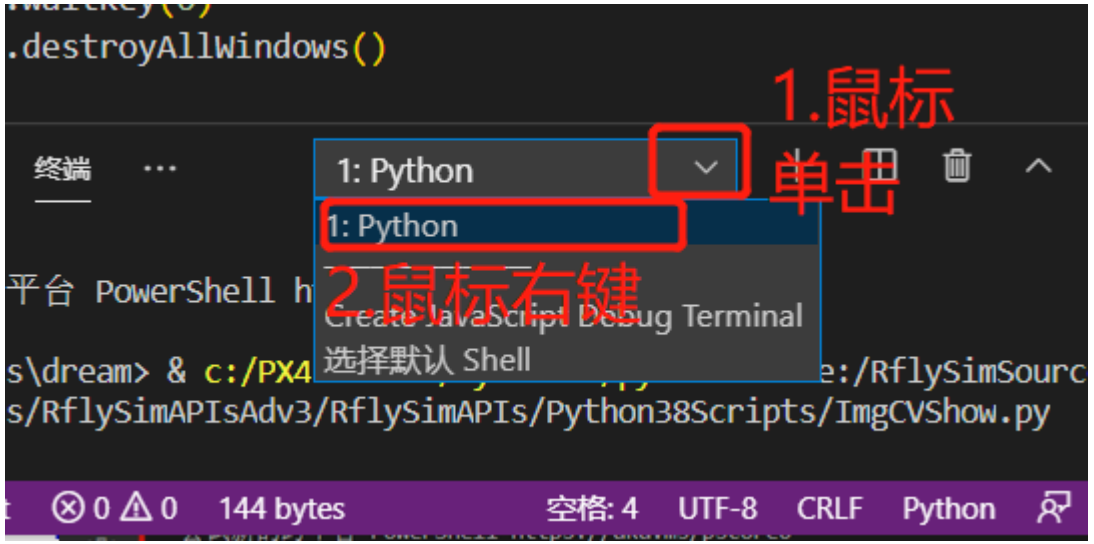


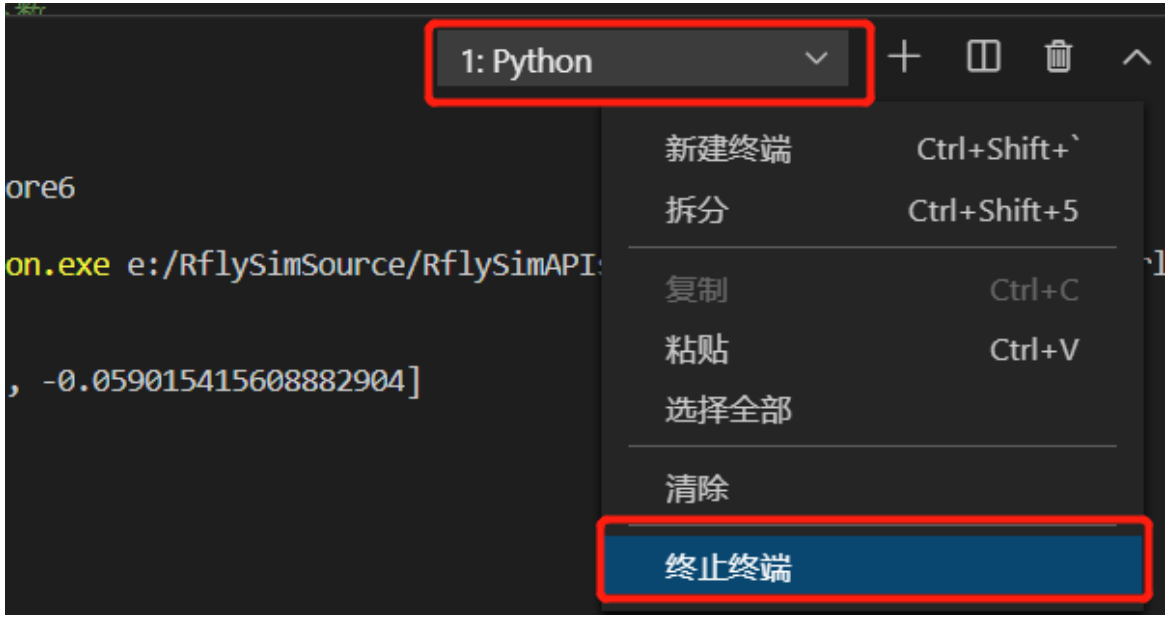


1.安装使用方法

1.5 VS Code Python程序结束运行

- 如左图所示，点击右下侧的Python按钮旁的下三角箭头展开显示所有终端窗口，在当前终端窗口条目（如下图中是1:Python）按下鼠标右键。如右图所示，在弹出菜单中点击“终止终端”，即可终止Python程序的运行。

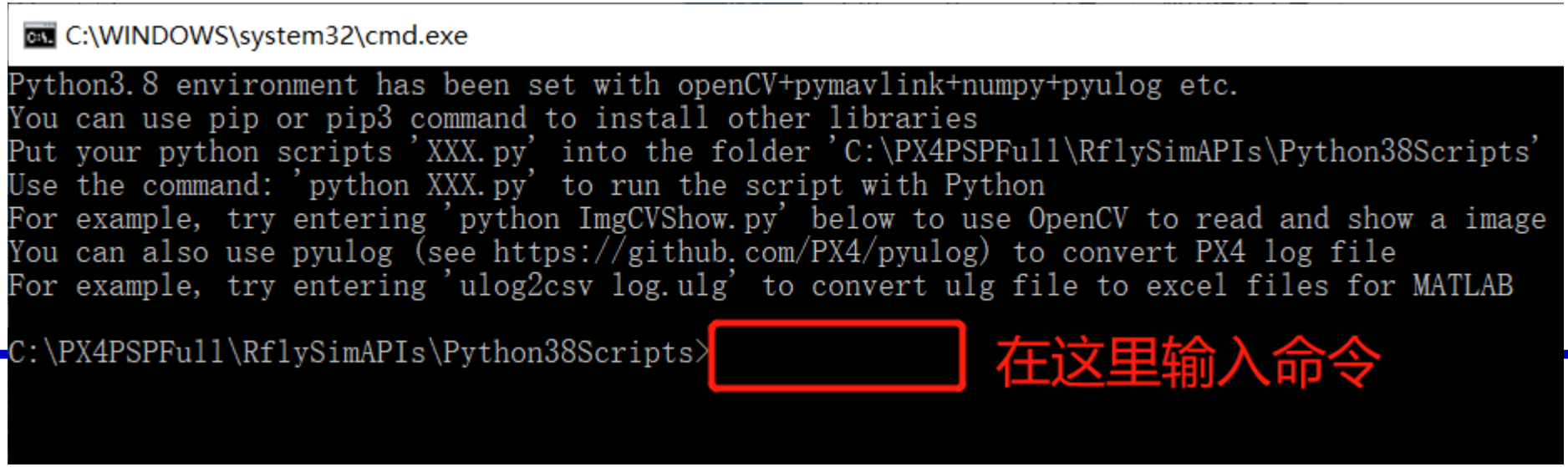




1.安装使用方法

1.6 Python38Env环境

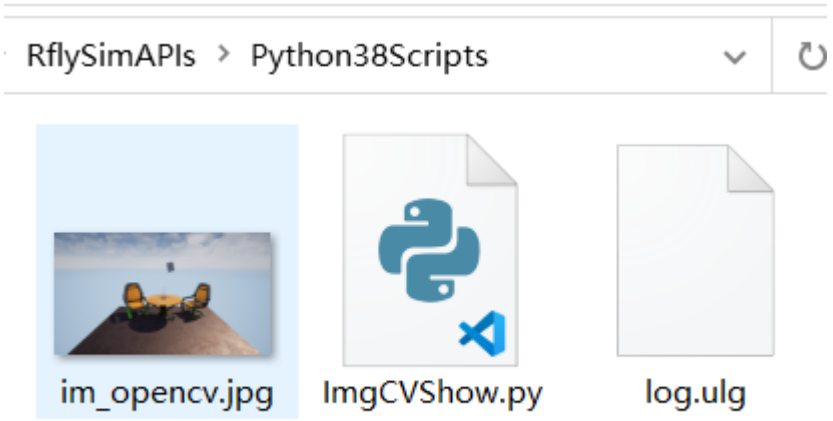
- 安装目录（默认是C:\PX4PSP）下的Python38文件夹包含了一个最新的Python运行环境，此环境已经包含了Opencv、pymavlink、pip等一系列工具，可以直接用于基础的无人机顶层控制算法的开发
- 本Python环境与Windows上安装的其他环境完全独立，不会影响其他Python环境或受它们的配置影响。
- 双击桌面“Python38Env”快捷方式或者双击“RflySimAPIs\Python38Env.bat”脚本，可以弹出下图终端窗口（已注册Python目录），可以调用Python环境

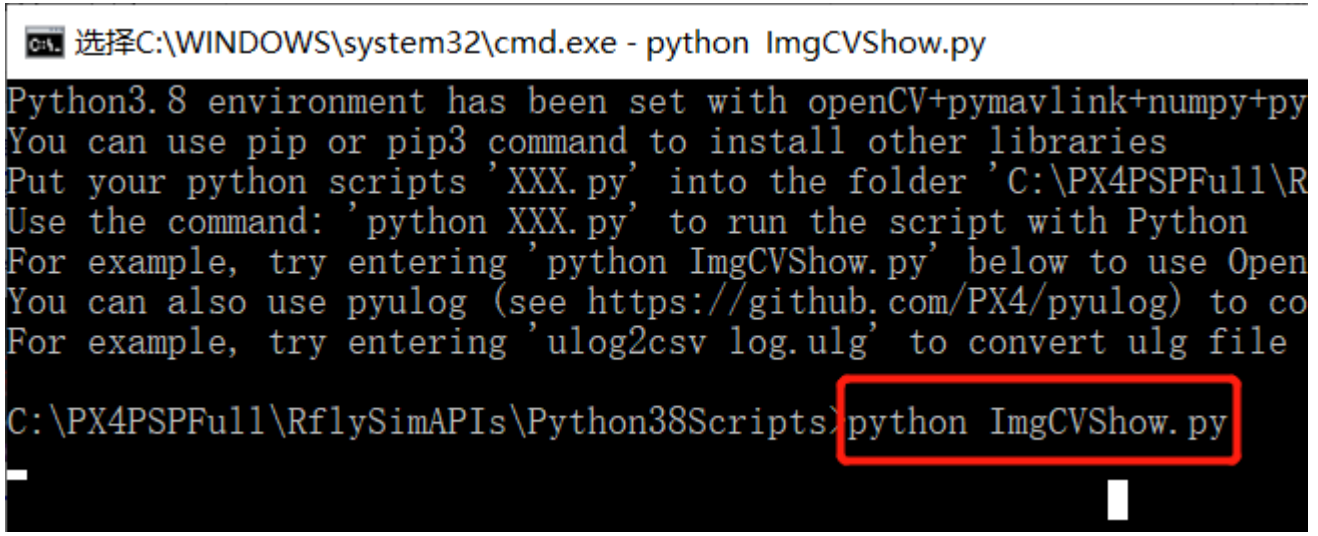


1.安装使用方法

1.7 Python38Env的使用

- 本Python环境的根目录为“RflySimAPIs\Python38Scripts”，将.py结尾的Python脚本文件***.py拷贝到本文件夹中，再执行命令“python ***.py”即可运行脚本
- 例如：输入命令“python ImgCVShow.py”可以和第1.4小节的例子一样，运行python脚本并打开一幅图像。

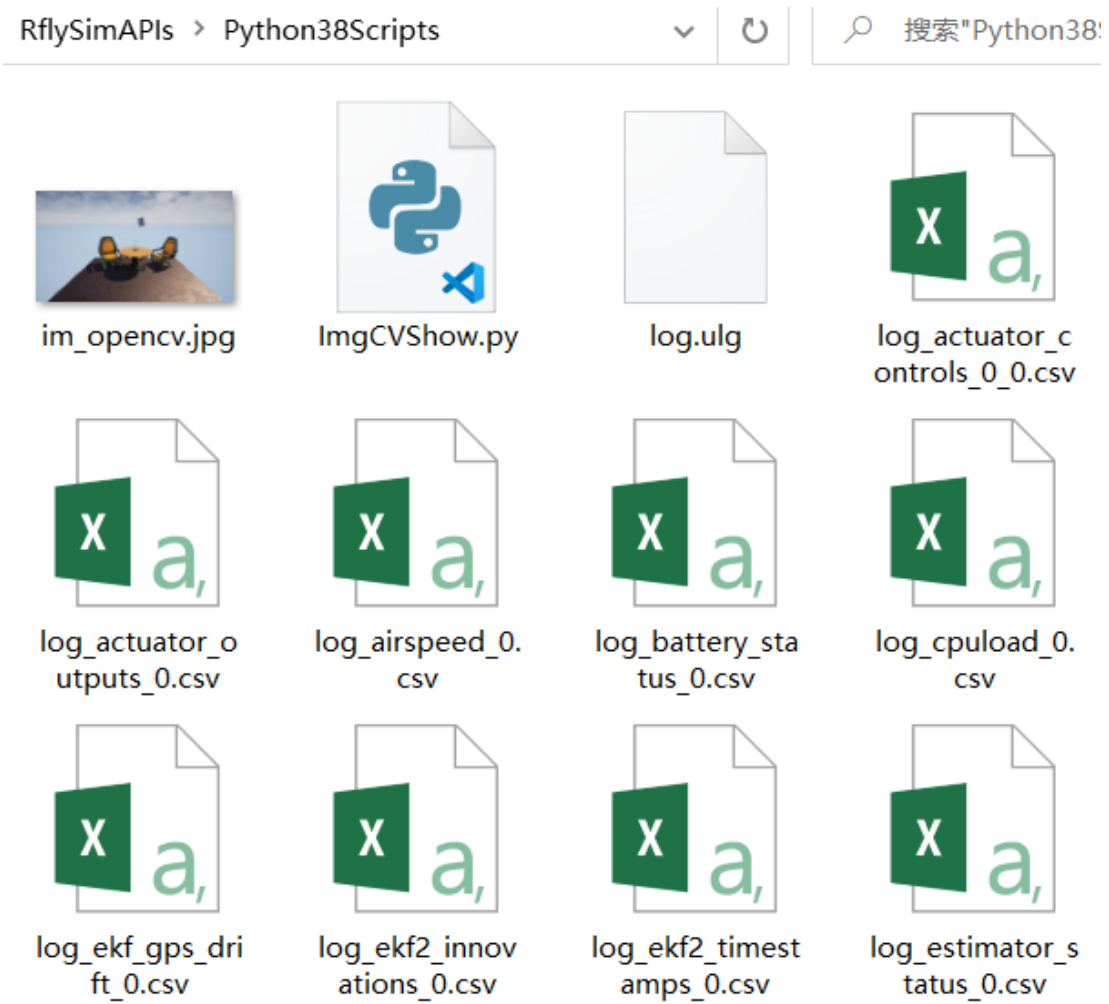




1.安装使用方法

1.8 Python38Env环境ulog日志读取

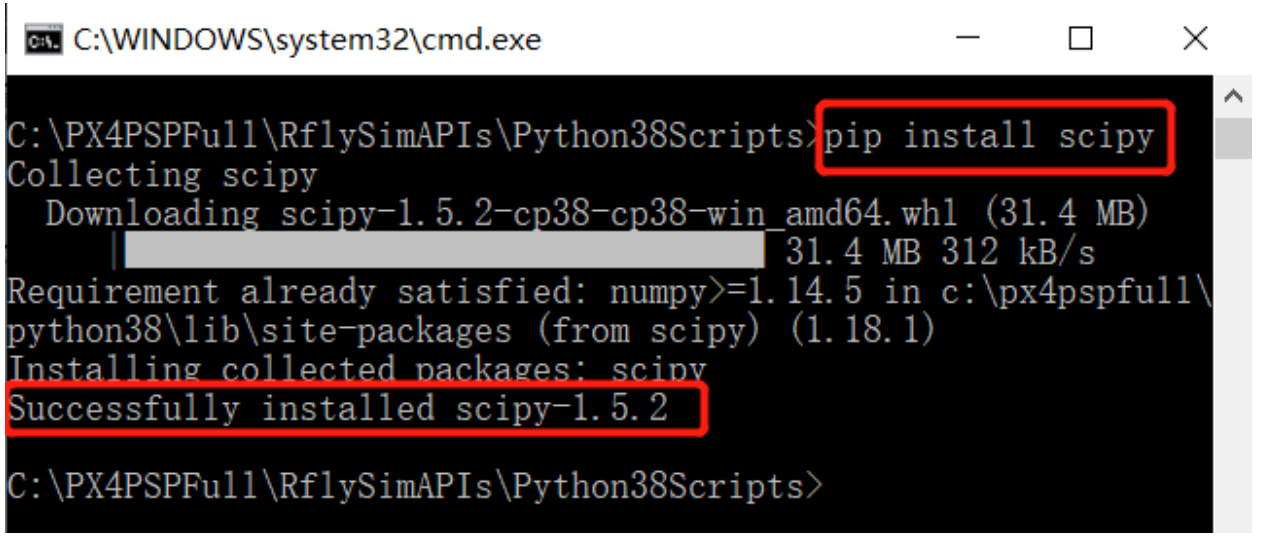
- 本Python环境已安装pyulog，可以用于解析PX4控制器的.ulg格式log日志并生成excel文件，可用于日志数据分析。
- 由于“RflySimAPIs\Python38Scripts”文件夹中已经存放了一个“log.ulg”文件，可以直接双击“RflySimAPIs\Python38Env.bat”来打开Python环境，并输入“ulog2csv log.ulg”，就可以得到日志文件
- 这些文件可以用Excel、MATLAB或其他统计分析软件打开来进行飞行数据分析。



1.安装使用方法

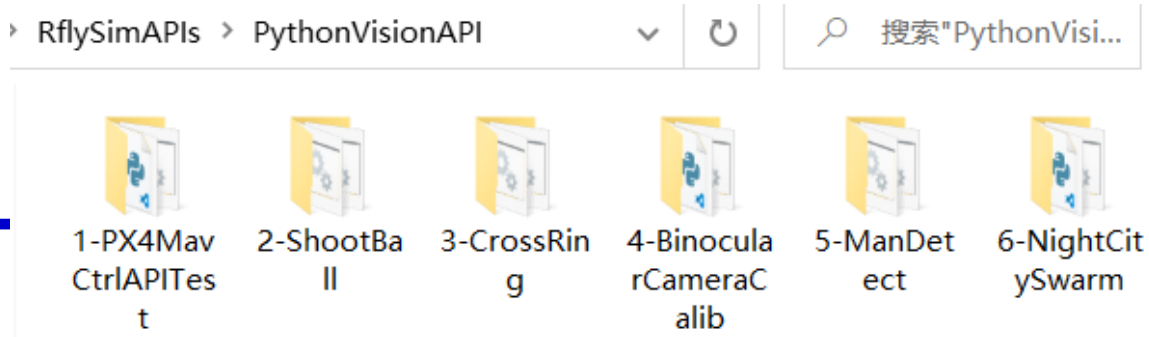
1.9 Python38Env功能扩展

- 本Python环境已安装openCV、pymavlink、numpy、pyulog等模块库，并支持安装其他更多组件。
- 其他组件的安装可以用pip install ***命令来安装
- 目前的Python常用库包括Scipy, Pillow, Matplotlib, Panda等。以安装Scipy为例，只需要双击“RflySimAPIs\Python38Env.bat”文件，在命令行中输入“pip install scipy”即可自动下载并完成安装
- 用户可以自行测试安装其他组件



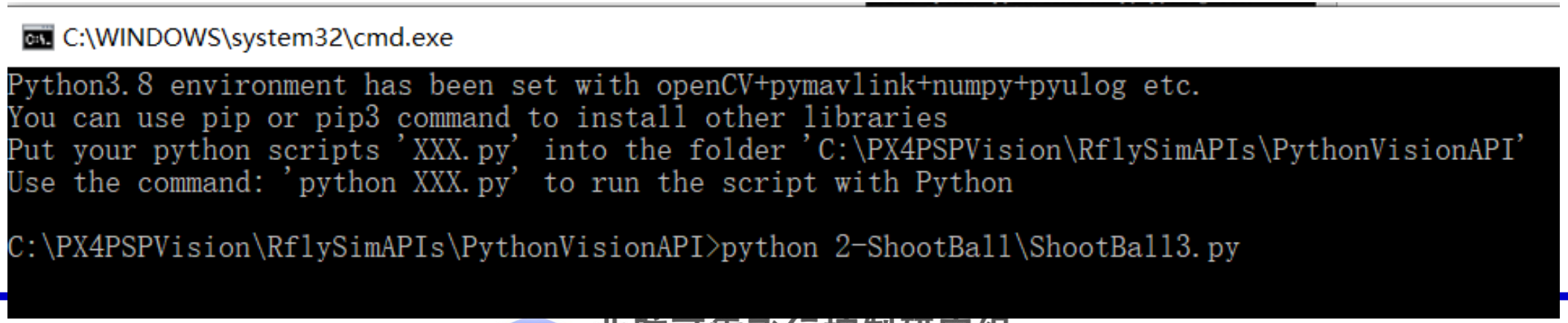
如果在线安装太慢（网络差或安装包体积大），可以去https://pypi.org/搜索并离线下载***.whl格式的安装包文件，将***.whl文件拷贝到“RflySimAPIs\Python38Scripts”目录，然后使用命令“pip install ***.whl”来安装

1.安装使用方法



1.10 视觉控制例程的运行

- 本讲课程的视觉控制例程都存放在“RflySimAPIs\PythonVisionAPI”路径下，这些例程可以采用VS Code来运行，也可以用Python38Env来运行。
- 设要运行“RflySimAPIs\PythonVisionAPI”目录下的“2-ShootBall\ShootBall3.py”例子脚本。第一种方法是用VS Code打开ShootBall3.py文件，然后按照第1.4小节方法直接运行。
- 第二种方法，双击打开“RflySimAPIs\PythonVisionAPI\ Python38Run.bat”文件，在弹出窗口中输入：“python 2-ShootBall\ShootBall3.py”，即可运行本脚本



1.安装使用方法

1.11 视觉仿真难点与RflySim平台解决方式

- 问题一：如何实现高速取图？
  解决方案：不在UE4程序内部取图（Airsim采用此方式，开启图像采集会严重掉帧），采用Python/C/Simulink直接读取RflySim3D的图像，减少中间环节，720P多窗口图像读取消耗用时间在5ms以内（200Hz以上），且不会干扰UE4渲染效率
- 问题二：如何实现多目相机的获取
  解决方案：支持开启任意个RflySim3D窗口，每个窗口可以独立配置显示视角（机载相机或者地面观察视角等）
- 问题三：如何配置分辨率、相机位置、几号飞机取图？
  解决方案：支持通过键盘快捷键调整，也支持通过UDP发送指令来控制RflySim3D的视角显示参数。
- 问题四：如何保证仿真结果可直接用于真机？
  解决方案：我们提供的控制接口底层直接收发Mavlink数据，由于使用跨平台的Python语言，直接拷贝机载计算机就能使用。（后续会提供Simulink视觉接口，支持代码生成）

大纲

1. 安装使用方法
2. 基础接口使用
3. 单目视觉控制例子
4. 双目视觉控制例子
5. 小结

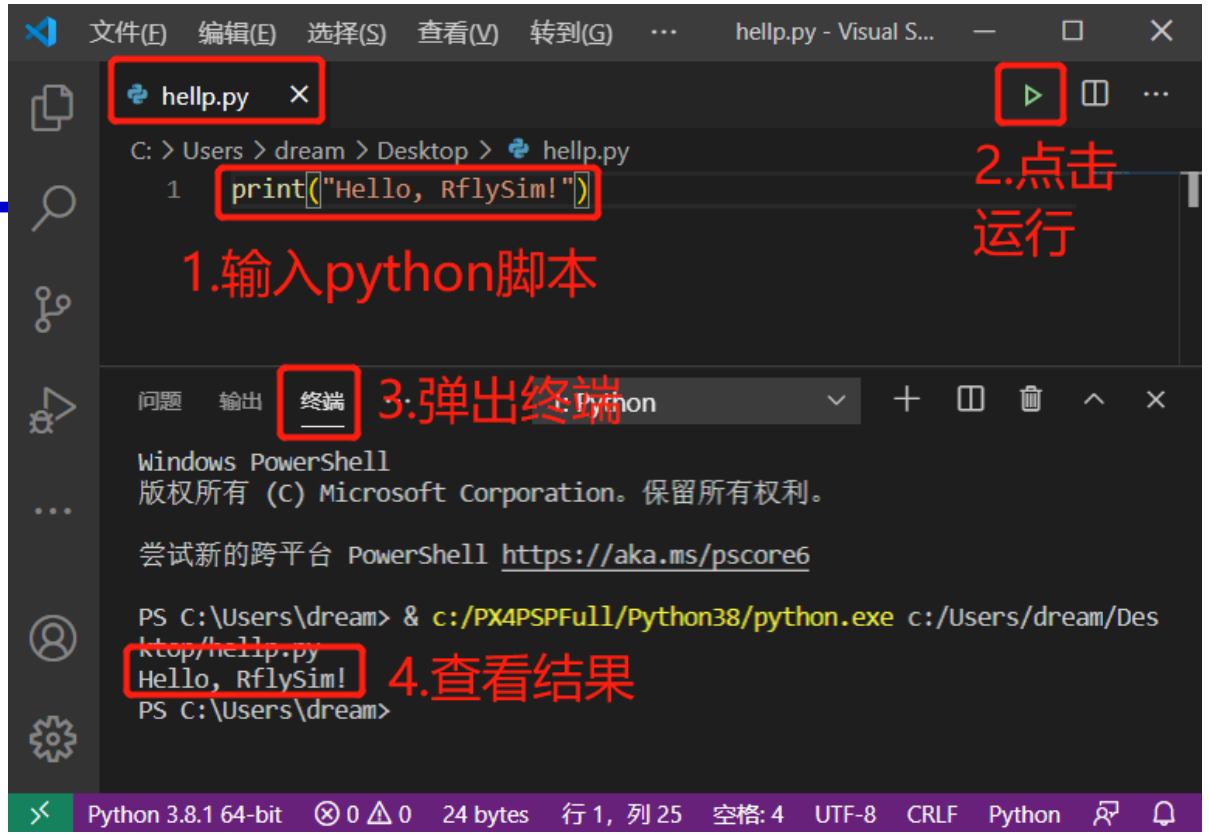
2.基础接口使用

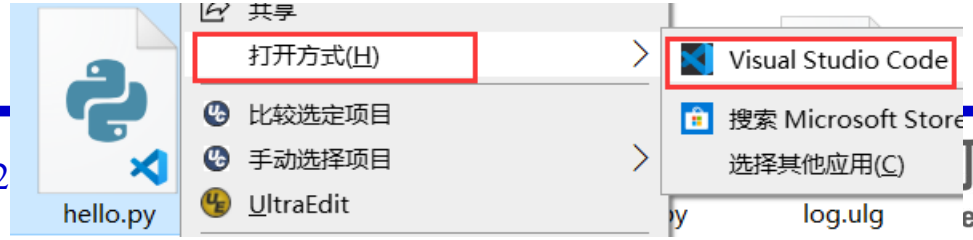
2.1 最简单的Python3文件运行测试

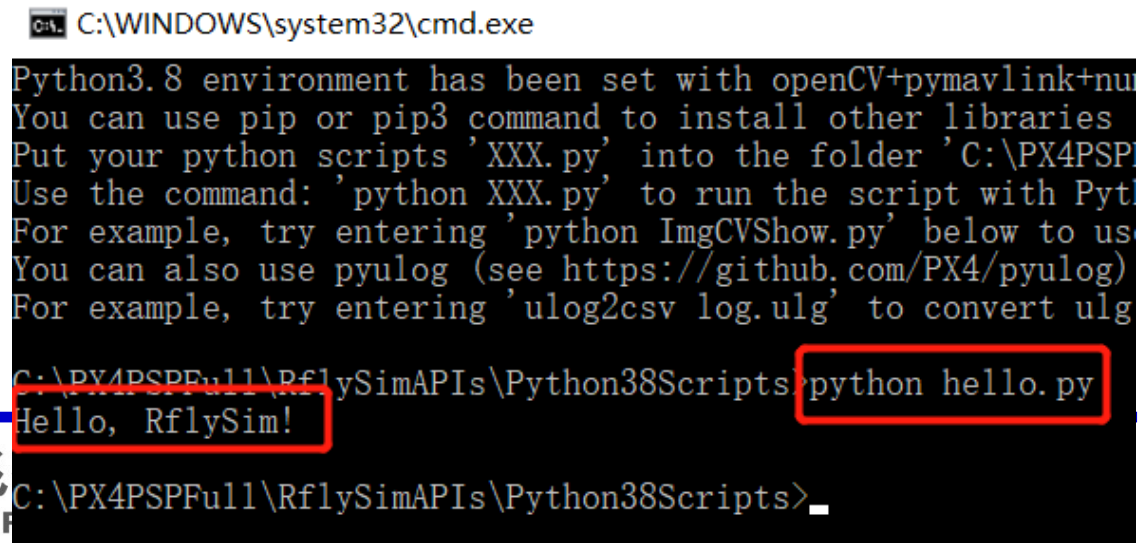
- 在“RflySimAPIs\Python38Scripts”文件夹内，新建一个txt文件，并重命名为hello.py
- 右键该文件，用VS Code打开，在其中敲入如下代码

```
print("Hello, RflySim!")
```

- 在VS Code中运行，如右图查看结果
- 同理，双击Python38Env桌面快捷方式或“RflySimAPIs\Python38Env.bat”，如下图查看运行结果。







2.基础接口使用

2.2 基础Python3语法学习

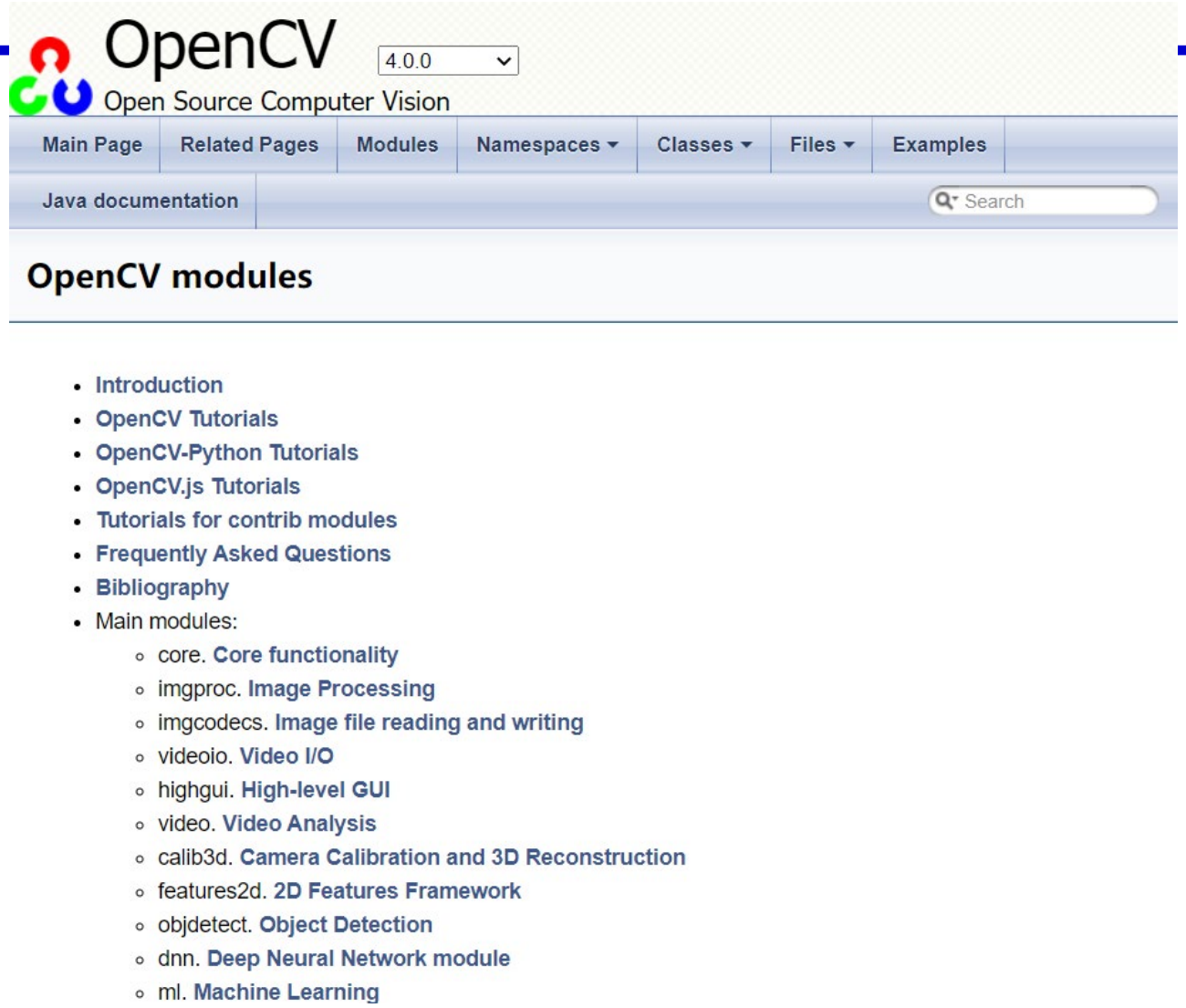
- 访问：https://runoob.com/python3
  了解学习Python3的基础知识与编程方法（与MATLAB语言相似）上手容易
- 注：Python2目前已经停止更新，推荐大家直接学习Python3进行开发
- Python3默认使用UTF-8编码，因此原生支持中文字符
- Python采用缩进的方式来区分代码层级，因此编写代码是要注意缩进（可在VS Code中安装Guides, indent-rainbow等插件来辅助）



2.基础接口使用

2.3 基础Python-OpenCV学习

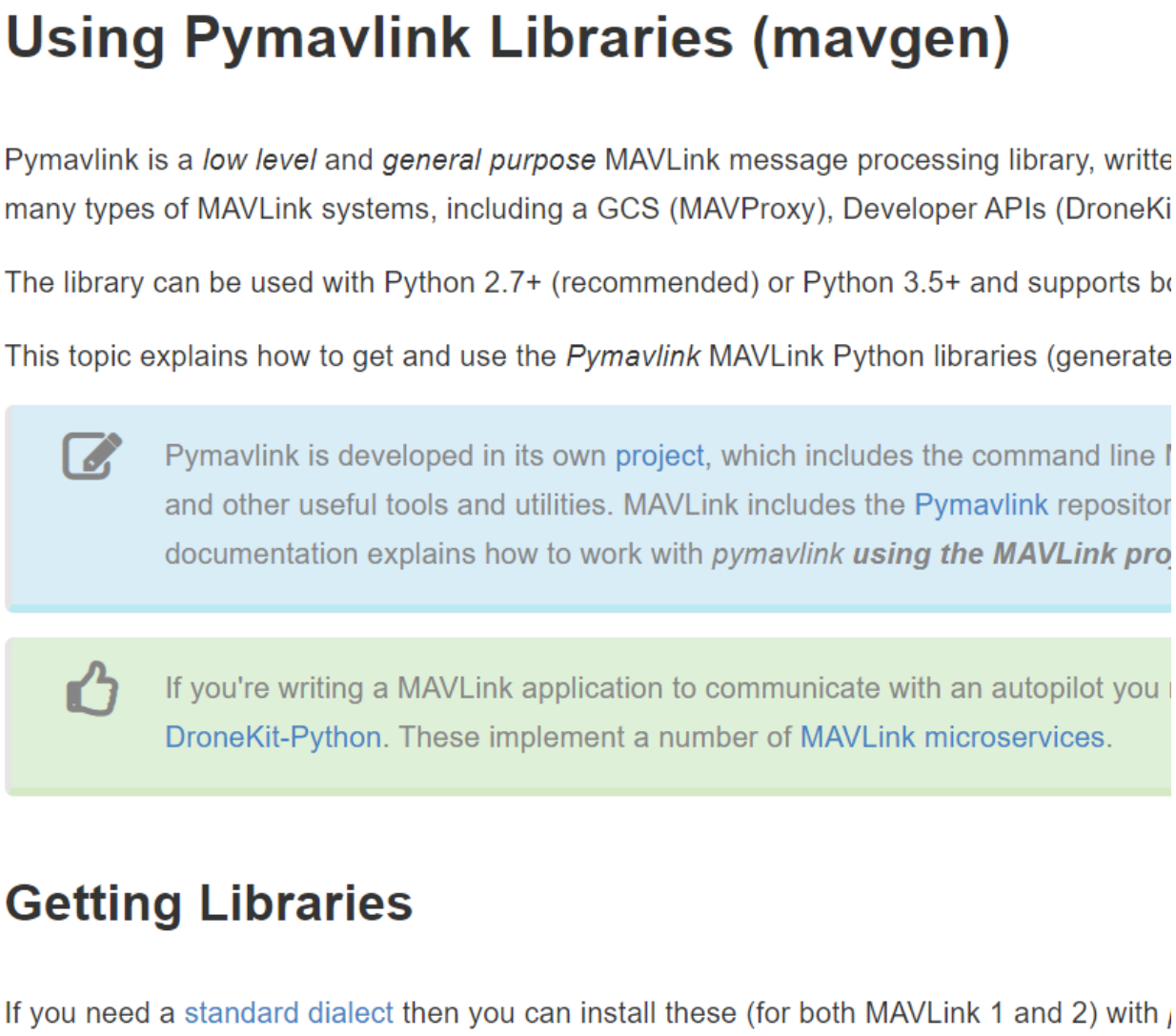
- OpenCV是一个开源的跨平台计算机视觉和机器学习软件库，应用广泛
- 官方教程网址：
- https://docs.opencv.org/4.0.0/
- 推荐中文翻译文档网址：
- https://github.com/makelove/OpenCV-Python-Tutorial
- 或直接打开“RflySimAPIs\PythonVision API\ OpenCV-Python-Tutorial-zh.pdf”来学习



2.基础接口使用

2.4 Pymavlink的学习

- Pymavlink是MAVLink通信协议的Python版本，目前已经预装在平台Python环境中，通过它可以方便的通过串口、UDP、TCP等方式与Pixhawk真机通信，进行顶层控制
- 官方文档网址如下：https://mavlink.io/en/mavgen_python
- 请自行学习其详细使用方法
- 注：Pymavlink只是一个方便使用的库函数，如果有更高的自定义需求，需要学习MAVLink协议的使用方法



Using Pymavlink Libraries (mavgen)

Pymavlink is a *low level* and *general purpose* MAVLink message processing library, writte
many types of MAVLink systems, including a GCS (MAVProxy), Developer APIs (DroneKi

The library can be used with Python 2.7+ (recommended) or Python 3.5+ and supports b

This topic explains how to get and use the *Pymavlink* MAVLink Python libraries (generate

Pymavlink is developed in its own project, which includes the command line
and other useful tools and utilities. MAVLink includes the Pymavlink repositor
documentation explains how to work with *pymavlink* ***using the MAVLink pro***

If you're writing a MAVLink application to communicate with an autopilot you
DroneKit-Python. These implement a number of MAVLink microservices.

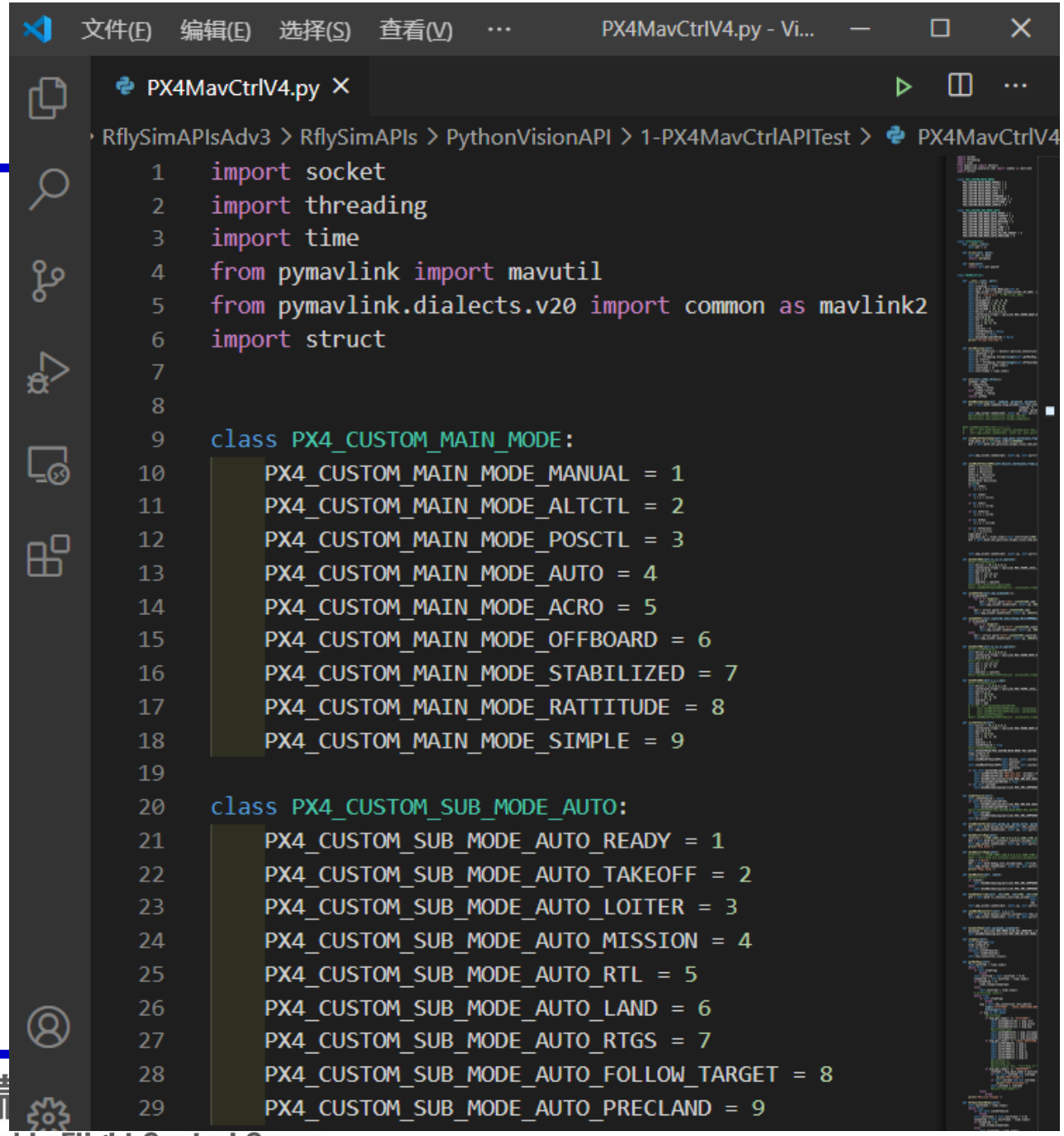
Getting Libraries

If you need a standard dialect then you can install these (for both MAVLink 1 and 2) with

2.基础接口使用

2.5 RflySim中Python控制接口介绍

- “RflySimAPIs\PythonVisionAPI\1-PX4MavCtrlAPITest\PX4MavCtrlV4.py”文件是RflySim平台的Python控制接口文件
- 本接口包含了MAVLink消息的收发、UE4场景控制、Pixhawk的Offboard控制等接口。
- 本接口底层数据为MAVLink格式，可以连接RflySim系统进行软/硬件在环仿真，也可以连接真实Pixhawk硬件（通过串口或UDP网络）进行真机的控制。



2.基础接口使用

```
self.uavAngEular = [0, 0, 0]
self.uavAngRate = [0, 0, 0]
self.uavPosNED = [0, 0, 0]
self.uavVelNED = [0, 0, 0]
```

Pixhawk实时状态数据

2.6 PX4MavCtrlV4内部原理

class PX4_CUSTOM_MAIN_MODE：#PX4主模块枚举变量，用于设置模式

class PX4_CUSTOM_SUB_MODE_AUTO: #PX4子模块枚举变量

class PX4MavCtrler: # RflySim的主要通信接口类

def InitMavLoop: #启用MAVLink接收线程，随时接收并更新MAVLink消息

def sat: #一个饱和函数，用于控制变量的限幅

def SendMavCmdLong：#发送MAVLink消息的COMMAND_LONG消息

def sendMavOffboardCmd：# 发送Offboard命令给飞控，使其进入Offboard模式

def sendMavOffboardAPI：# 更新Offboard消息的数据（该数据会以一定频率发送）

def SendVelNED：# 发送地球坐标系速度指令

def sendUE4Cmd：# 发送命令给UE4，控制UE4的显示

def sendUE4Pos： # 发送一个物体的三维坐标给UE4，使显示一个物体



2.基础接口使用

2.6 PX4MavCtrlV4内部原理

def SendVelFRD: #发送机体速度

def SendPosNED: #发送NED位置，让飞机飞到指定位置（相对解锁点）

def initOffboard: # 初始化Offboard模式

def endOffboard: # 结束Offboard模式

def sendMavSetParam: # 发送MAVLink消息改变Pixhawk参数

def SendHILCtrlMsg : #发送rfly_msg消息到飞控（见第3讲4.3节内容）

def SendMavArm : #发送解锁命令

def SendRcOverride : #发送并模拟遥控器信号

def sendMavManualCtrl : #发送并模拟归一化遥控器信号

def SendSetMode : #发送并设置Pixhawk模式

def stopRun : #停止运行MAVLink数据接收线程

def getMavMsg : #更新MAVLink接收的数据

2.基础接口使用

2.7 接口使用例子

- “1-PX4MavCtrlAPITest\PX4MavCtrlAPITest.py”是一个接口使用的Python例子，具体代码解析如下：

```
#新建一个Mavlink通信实例，CopterSim接口是20100
mav = PX4MavCtrl.PX4MavCtrler(20100)

# RflySim3D样式调整API: sendUE4Cmd函数
# 样式mav.sendUE4Cmd(cmd,windowID=-1)，其中cmd应该输入字符串
# 向RflySim3D发送窗口调整命令，cmd为具体命令字符串，windowID为接收的窗口号(如果同时打开多个RflySim3D样式调整API)，-1表示发送给所有窗口
# RflyChangeMapbyName命令表示切换地图，后面跟的字符串为地图名字，这里将所有打开窗口切换为草地地图
mav.sendUE4Cmd(b'RflyChangeMapbyName Grasslands
```

PX4MavCtrlAPITest.py

C: > PX4PSPFull > RflySimAPIs > PythonVisionAPI > 1-PX4MavCtrlAPIT

```
1  import time
2  import math
3  import sys
4  import PX4MavCtrlV4 as PX4MavCtrl
5
6
7  #新建一个Mavlink通信实例，CopterSim接口是20100
8  mav = PX4MavCtrl.PX4MavCtrler(20100)
9
10
```

导入库

新建一个接口实例



2.基础接口使用

2.7 接口使用例子

```
# RflySim3D生成三维物体并控制位姿姿态API: sendUE4Pos函数
# 样式mav.sendUE4Pos(CopterID, VehicleType, RotorSpeed, PosM, AngEulerRad, windowsID=0)
# 向RflySim3D发送并生成一个三维物体，其中：物体ID为CopterID=100；
# 飞机类型VehicleType=30（人物）；旋翼转速RotorSpeed=0RPM；位置坐标PosM=[2.5,0,-8.086]m
# 飞机姿态角AngEulerRad=[0,0,math.pi]rad （旋转180度面对飞机）,接收窗口号默认windowsID=-1（发给所有打开的RflySim3D程序）
# VehicleType：3四旋翼，5/6六旋翼，30人物，40标定棋盘格，50/51车，球状60发光灯，100飞翼固定翼，150/152环形方形靶标
mav.sendUE4Pos(100,30,0,[2.5,0,-8.086],[0,0,math.pi])

#发送消息，使所有场景中的CopterID=100（刚创建的人物）变成行走的人的样式
# RflyChange3DModel命令后跟 飞机ID + 期望样式，这里12的样式是行走的人
mav.sendUE4Cmd(b'RflyChange3DModel 100 12')
```



2.基础接口使用

2.7 接口使用例子

```
# 命令RflyChangeViewKeyCmd表示模拟RflySim3D的快捷键操作，B 1快捷键表示将焦点切换到CopterID=1的对象
# 这里设定向0号窗口发送，其他窗口不发送
mav.sendUE4Cmd(b'RflyChangeViewKeyCmd B 1',0)

# V 1快捷键表示将焦点切换到CopterID=1的对象
mav.sendUE4Cmd(b'RflyChangeViewKeyCmd V 1',0)

# RflyCameraPosAng x y z roll pith yaw 设置相机相对机体中心的位置方向，默认给0
# 这里设置前置相机的位置为[0.1 -0.25 0]
mav.sendUE4Cmd(b'RflyCameraPosAng 0.1 0 0',0)

# r.setres 720x405w 是UE4内置的命令，表示切换分辨率到720x405
mav.sendUE4Cmd(b'r.setres 720x405w',0)
```



2.基础接口使用

2.7 接口使用例子

```
# 向1号窗口发送快捷键命令，将焦点切换到飞机1
mav.sendUE4Cmd(b'RflyChangeViewKeyCmd B 1',1)

# 向0号窗口发送快捷键控制命令，N 1 快捷键表示将视角切换到地面固定视角1
mav.sendUE4Cmd(b'RflyChangeViewKeyCmd N 1',1)

# 这里设置相机的位置为[-2 0 -9.7]
mav.sendUE4Cmd(b'RflyCameraPosAng -2 0 -9.7',1)

#开启Mavlink监听CopterSim数据，并实时更新，数据见右下图
mav.InitMavLoop()

#显示从CopterSim收到的位置信息
print(mav.uavPosNED)
```

```
self.uavAngEular = [0, 0, 0]
self.uavAngRate = [0, 0, 0]
self.uavPosNED = [0, 0, 0]
self.uavVelNED = [0, 0, 0]
```

Pixhawk实时状态数据

2.7 接口使用例子

```
#开启Offboard模式
mav.initOffboard()
#发送期望位置信号，飞到目标点 0,0，-1.7位置，偏航角为0
mav.SendPosNED(0, 0, -1.7, 0)
#发送解锁命令
mav.SendMavArm(True)

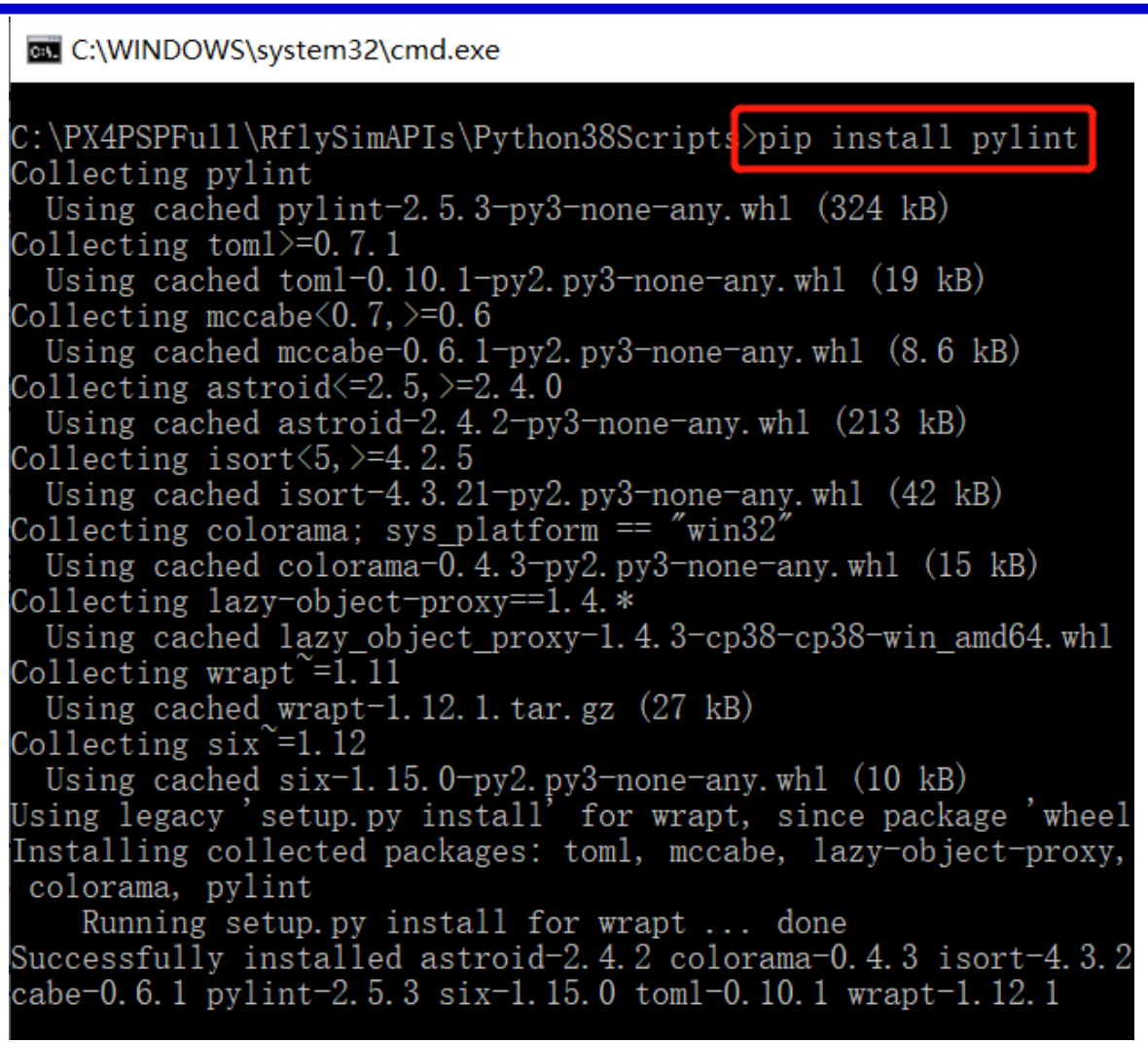
#发送期望速度信号，0.2m/s向下降落，z轴向下为正
mav.SendVelNED(0, 0, 0.2, 0)
#退出Offboard控制模式
mav.endOffboard()

#退出Mavlink数据接收模式
mav.stopRun()
```

2.基础接口使用

2.8 VS Code Python调试功能开启

- 为了方便观察每一行Python代码的运行结果，这里需要开启Python的调试功能，配置流程如下：
- 双击“RflySimAPIs\Python38Env.bat”打开Python环境，并输入指令“pip install pylint”来安装Python检查器pylint，安装成功的结果如右图
- 关闭窗口，开始用VS Code进行代码调试



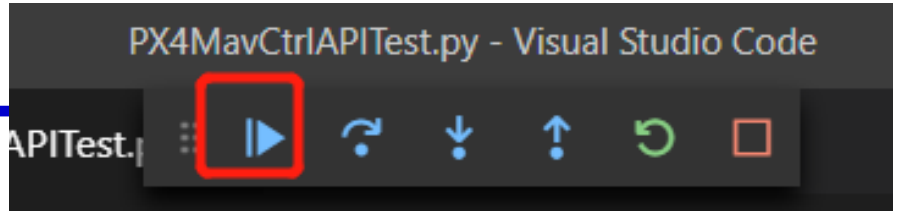
2.基础接口使用

2.9 接口使用例子

- 在Windows资源管理器中定位到“RflySimAPIs\PythonVisionAPI\1-PX4MavCtrlAPITest”文件夹
- 双击“PX4MavCtrlAPITest.bat”脚本来打开一个飞机的PX4 SITL仿真系统
- 用VS Code打开“PX4MavCtrlAPITest.py”文件，如右图在每条关键语句前面点上断电（红点），按下图所示开启调试模式，点右下图箭头按钮，依次执行语句



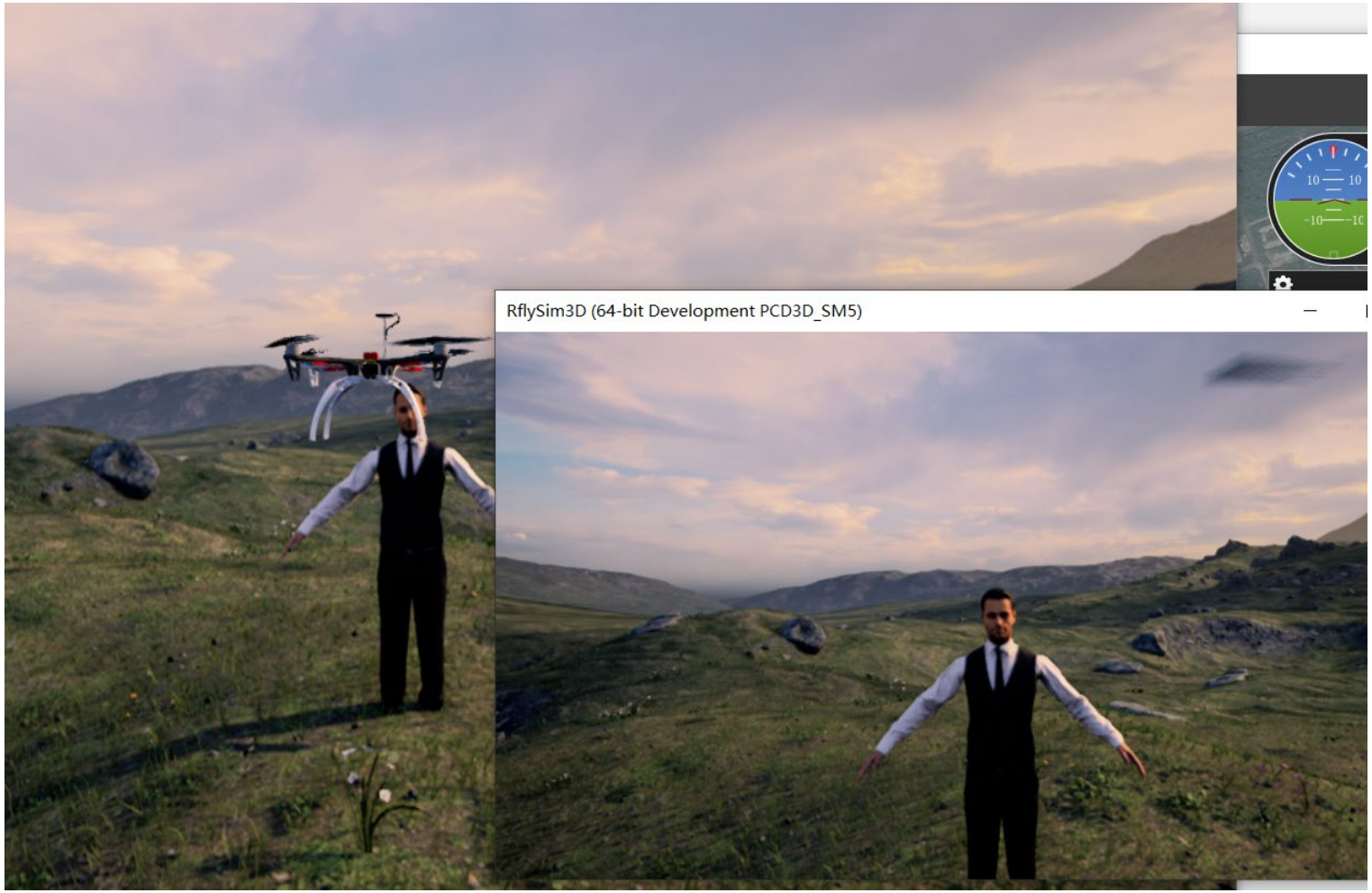




2.基础接口使用

2.10 接口例子实验效果

- 本例程的现象是，python程序发送一系列指令，在RflySim3D程序中新建了一个走动的人的目标，设置了视角形式、尺寸、位置，向仿真的无人机发送控制指令使其起飞与降落。
- 如右图所示，本例子会打开两个RflySim3D窗口，一个是前置摄像头，一个是上帝视角观测



2.基础接口使用

- 2.10 接口例子结束仿真
- 在下图所示“PX4MavCtrlAPITest.bat”脚本开启的命令提示符窗口中，按下回车键（任意键）就能快速关闭CopterSim、QGC、RflySim3D等所有程序。
- 如右下图，在VS Code中按本讲1.5节步骤，点击“终止终端”，退出脚本运行
- 关闭“PX4MavCtrlAPITest.py”打开的VS Code窗口





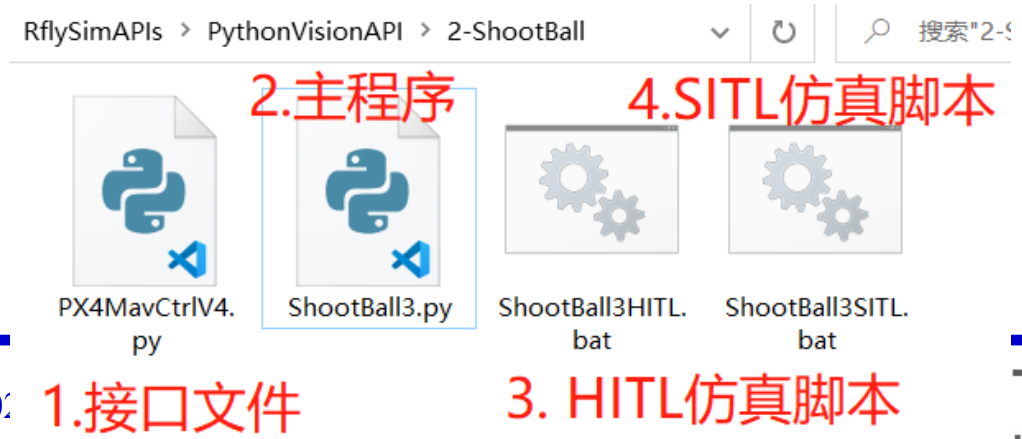
大纲

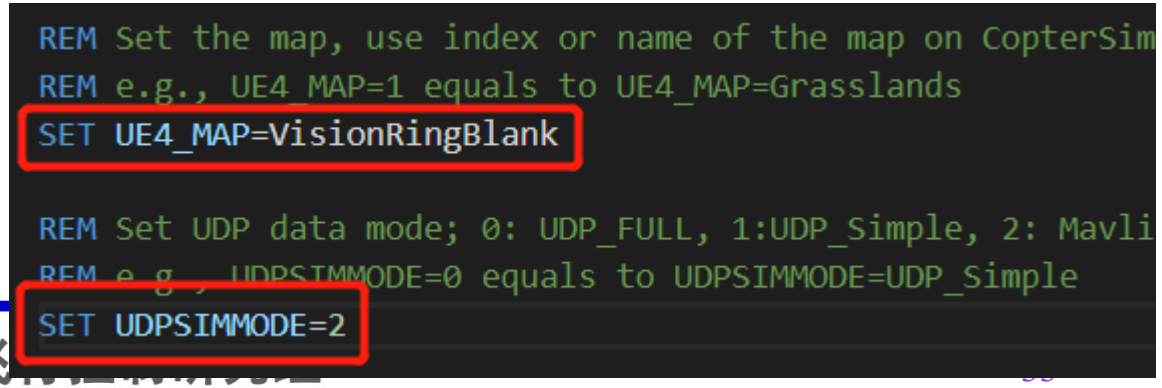
1. 安装使用方法
2. 基础接口使用
3. 单目视觉控制例子
4. 双目视觉控制例子
5. 小结

3.单目视觉控制例子

3.1 无人机撞击小球实验

- 在Windows资源管理器中，打开并进入“RflySimAPIs\PythonVisionAPI\2-ShootBall”文件夹，其中的内容如下图
- 其中，“PX4MavCtrlV4.py”是上一节介绍的接口文件，“ShootBall3.py”是本例程的主Python程序，“ShootBall3HITL.bat”是快速启动硬件在环仿真的脚本，“ShootBall3SITL.bat”是快速启动软件在环仿真的脚本。如右下图所示，后两者相对于桌面的S/HITLRun快捷方式的区别在于：“UE4_MAP”地图场景变量选择了用于视觉的平坦草地场景“VisionRingBlank”；其次，“UDPSIMMODE”通信UDP模式选择了“Mavlink_Full”模式，便于与Python通信；最后打开2个RflySim3D窗口





3.单目视觉控制例子

3.2 撞击小球实验代码解析

- 用VS Code打开“ShootBall3.py”文件，

ShootBall3.py

C: > PX4PSPFull > RflySimAPIs > PythonVisionAPI > 2-ShootBall > ShootBall3.py

```python
import time
import mmap
import numpy as np
import cv2.cv2 as cv2
from pymavlink.dialects.v20 import common as mavlink2

import win32gui, win32ui, win32con
from ctypes import windll

import sys
import PX4MavCtrlV4 as PX4MavCtrl

print(sys.executable)
isEmptyData = False
mav = PX4MavCtrl.PX4MavCtrler(20100)
mav.InitMavLoop()

lastTime = time.time()
startTime = time.time()
timeInterval = 0.01  #定时时钟，这里是0.01s执行一次
flag = 0
# parameters
width = 720
height = 405
channel = 4
min_prop = 0.000001
```

1.导入库并初始化变量

```python
def window_enumeration_handler(hwnd, window_hwnds):
    if win32gui.GetClassName(hwnd) == "UnrealWindow":
        window_hwnds.append(hwnd)


#设置空白草地地形，两个窗口都设置，第一个窗口用于摄像头图像读取，第二个窗口用于效果预览
mav.sendUE4Cmd(b'RflyChangeMapbyName VisionRingBlank')
time.sleep(1)
#新建一个球形，设置位置和坐标，使用默认样式红色
mav.sendUE4Pos(100,152,0,[3,0,-2],[0,0,0])
time.sleep(0.5)
#将焦点切换到飞机1
mav.sendUE4Cmd(b'RflyChangeViewKeyCmd B 1',0)
time.sleep(0.5)
#将视角切换到1视角（前置摄像头）
mav.sendUE4Cmd(b'RflyChangeViewKeyCmd V 1',0)
time.sleep(0.5)
#设置前置摄像头位置在相对机体中心[0.3,0,0.05]的地方
mav.sendUE4Cmd(b'RflyCameraPosAng 0.3 0 0.05 0 0 0',0)
time.sleep(0.5)
#设置前置摄像头画面为720x405
mav.sendUE4Cmd(b'r.setres 720x405w',0)
```

2.获取RflySim3D窗口句柄

3. 发送指令，设置RflySim3D的窗口样式（尺寸、障碍物位置、相机视角等）

3.单目视觉控制例子

3.2 撞击小球实验代码解析

```
time.sleep(2)
window_hwnds = []
win32gui.EnumWindows(window_enumeration_handler, window_hwnds)
if len(window_hwnds)>0:
    print(window_hwnds)
    window_hwnds.sort()
    hWnd = window_hwnds[0]
    print(hWnd)

#hWnd = win32gui.FindWindow("UnrealWindow", None)

left, top, right, bot = win32gui.GetClientRect(hWnd)
windows_width = right - left
windows_height = bot - top

if hWnd and windows_width != width:
    hWnd = window_hwnds.sort(reverse=True)
    hWnd = window_hwnds[0]
    left, top, right, bot = win32gui.GetClientRect(hWnd)
    windows_width = right - left
    windows_height = bot - top
    print("Not the desired window.")

if hWnd and windows_width == 0 and windows_height == 0:
    print("The UE4 window cannot be in minimum mode.")
    sys.exit(1)
```

4.获取RflySim窗口画布、尺寸等，便于图像获取

```
def procssImage():
    #这里处理图像，并获取到速度控制指令
    saveDC.SelectObject(saveBitMap)
    # 0-保存整个窗口，1-只保存客户区。如果PrintWindow成功函数返回值为1
    result = windll.user32.PrintWindow(hWnd, saveDC.GetSafeHdc(), 1)
    signedIntsArray = saveBitMap.GetBitmapBits(True)
    img_rgba = np.frombuffer(signedIntsArray, dtype='uint8')
    img_rgba.shape = (windows_height, windows_width, 4)
    img_bgr = cv2.cvtColor(img_rgba, cv2.COLOR_RGBA2RGB)
    img_bgr = cv2.resize(img_bgr, (width, height))

    p_i = calc_centroid(img_bgr)
    #print(p_i)
    #cv2.imshow("img", img_bgr)
    #cv2.waitKey(1)

    ctrl = controller(p_i) #返回前进速度，向下速度，右偏航速度
    return ctrl
```

5.图像处理主程序（每0.01秒运行一次），获取图像，识别球体，并计算控制量

3.单目视觉控制例子

3.2 撞击小球实验代码解析

```python
def calc_centroid(img):
    """Get the centroid and area of green in the image"""

    #hue_image = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    low_range = np.array([0,0,80])
    high_range = np.array([100,100,255])
    th = cv2.inRange(img, low_range, high_range)
    dilated = cv2.dilate(th, cv2.getStructuringElement(
        cv2.MORPH_ELLIPSE, (3, 3)), iterations=2)
    cv2.imshow("dilated", dilated)
    cv2.waitKey(1)

    M = cv2.moments(dilated, binaryImage=True)
    if M["m00"] >= min_prop*width*height:
        cx = int(M["m10"] / M["m00"])
        cy = int(M["m01"] / M["m00"])
        return [cx, cy, M["m00"]]
    else:
        return [-1, -1, -1]
```

6.获取小球中心位置函数

```python
def sat(inPwm,thres=1):
    outPwm= inPwm
    for i in range(len(inPwm)):
        if inPwm[i]>thres:
            outPwm[i] = thres
        elif inPwm[i]<-thres:
            outPwm[i] = -thres
    return outPwm

def controller(p_i):
    # if the object is not in the image, search in clockwise
    if p_i[0] < 0 or p_i[1] < 0:
        return [0, 0, 0, 1]

    # found
    ex = p_i[0] - width / 2
    ey = p_i[1] - height / 2

    vx = 2 if p_i[2] < max_prop*width*height else 0
    vy = 0
    vz = K_z * ey
    yawrate = K_yawrate * ex

    return [vx, vy, vz, yawrate]
```

7.饱和函数

8.控制器（像素误差映射为位置误差，再计算控制速度）

3.单目视觉控制例子

3.2 撞击小球实验代码解析

```
ctrlLast = [0,0,0,0]
while True:
    lastTime = lastTime + timeInterval
    sleepTime = lastTime - time.time()
    if sleepTime > 0:
        time.sleep(sleepTime)
    else:
        lastTime = time.time()
    #以下代码0.01s执行一次

    if time.time() - startTime > 5 and flag == 0:
        #下列代码第10s的时触发一次，发送一个解锁信号，并设定一个前飞目标
        print("5s, Arm the drone")
        mav.initOffboard()
        flag = 1
        mav.SendMavArm(True) #解锁命令
        print("Arm the drone!, and fly to NED 0,0,-5")
        mav.SendPosNED(0, 0, -5, 0) #飞到目标点 0,0, -5位置，命令范围15s
```

9.主程序while死循环

10.通过sleep函数实现定时器，0.01s一次循环

11.通过if语句构建一个状态机，第一个状态

```
if time.time() - startTime > 15 and flag == 1:
    flag = 2
    #以下脚本在30s时触发一次
    mav.SendPosNED(-30,-5, -5, 0)  # 速度为0，保持悬停
    print("15s, fly to pos: -30,-5, -5")

if time.time() - startTime > 25 and flag == 2:
    flag = 3
    print("25s, start to shoot the ball.")

if time.time() - startTime > 25 and flag == 3:
    ctrlNow = procssImage()
    ctrl = sat(ctrlNow,5)
    #以下代码添加一个惯性环节，避免x,y方向控制量变化太快
    if ctrl[0]-ctrlLast[0] > 0.5:
        ctrl[0]=ctrlLast[0]+0.05
    elif ctrl[0]-ctrlLast[0] < -0.5:
        ctrl[0]=ctrlLast[0]-0.05
    if ctrl[1]-ctrlLast[1] > 0.5:
        ctrl[1]=ctrlLast[1]+0.05
    elif ctrl[1]-ctrlLast[1] < -0.5:
        ctrl[1]=ctrlLast[1]-0.05
    ctrlLast = ctrl
    #print(isEmptyData)
    if not isEmptyData:
        mav.SendVelFRD(ctrl[0], ctrl[1], ctrl[2], ctrl[3])
```

12. 第15秒开始起飞，并飞到后方30m的位置

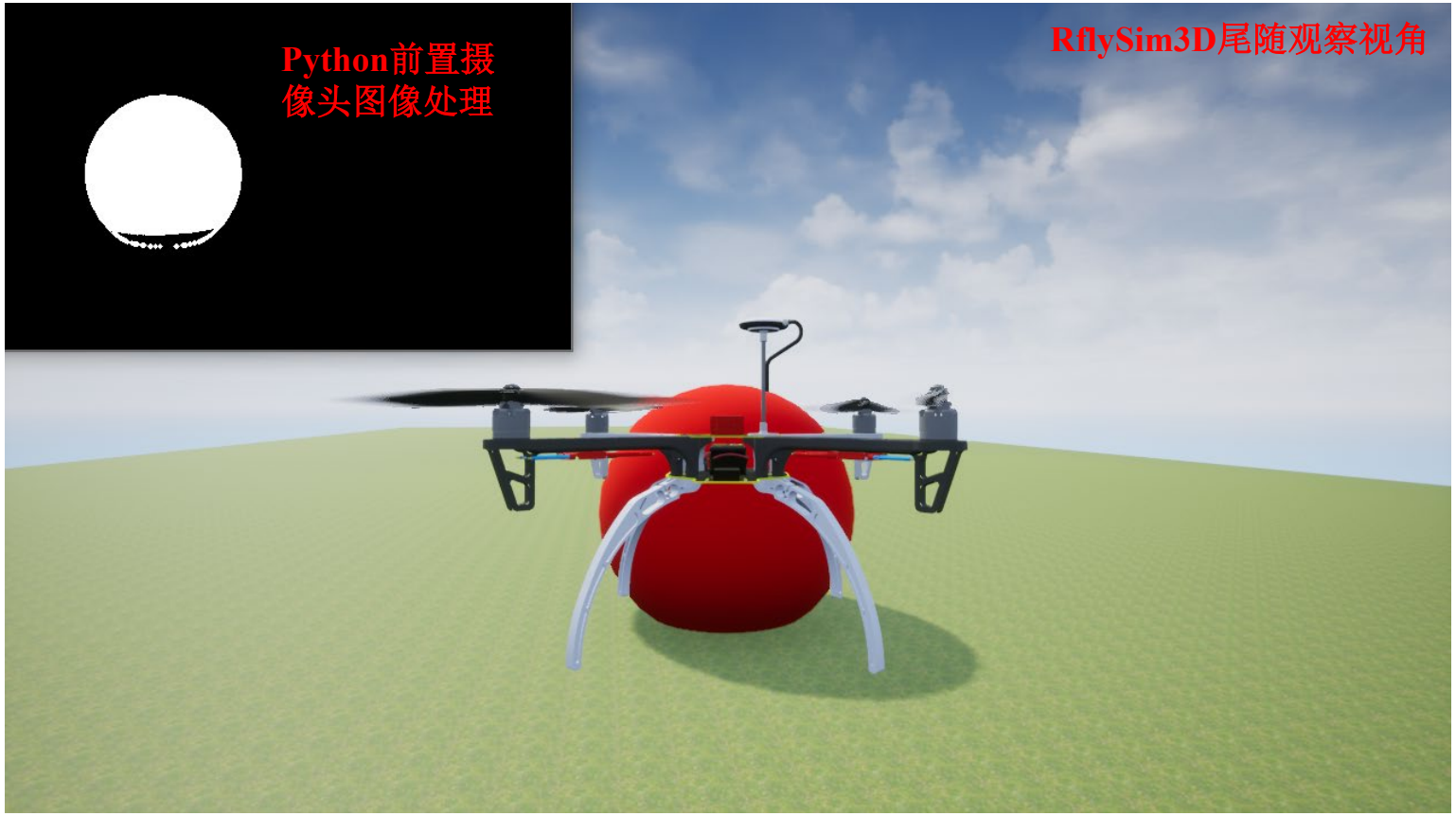
13. 第25秒开启视觉算法

14. 图像处理并计算控制量，使飞机飞向（带速度饱和控制）小球

3.单目视觉控制例子

3.3 撞击小球实验运行效果

- 双击运行“ShootBall3SITL.bat”文件开启软件在环仿真系统，再运行“ShootBall3.py”程序。在前方生成一个红色球体，让飞机飞到靠左后方一段距离，并开启视觉跟踪，飞到小球面前停止。
- 若要使用硬件在环仿真，设置好飞控后，运行“ShootBall3HITL.bat”脚本，并输入飞控串口号，来开启硬件在环仿真系统。

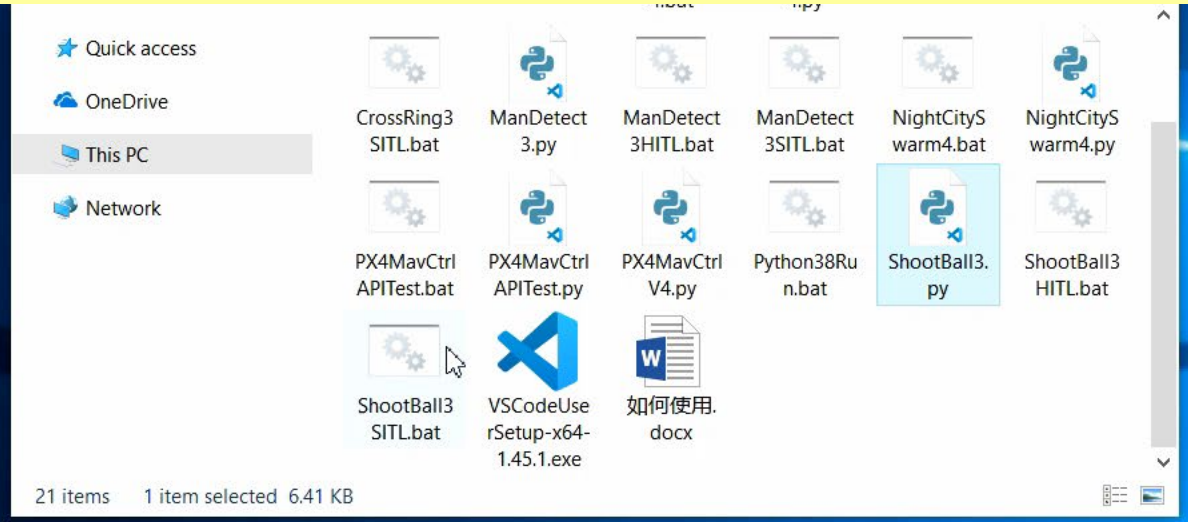


RflySim. 多旋翼无人机进行视觉导航控制 —— 撞击小球实验

本视频观看地址：

优酷： https://v.youku.com/v_show/id_XNDcwNjA4NTYwNA==.html

YouTube： https://youtu.be/PvxEfY7oMq4



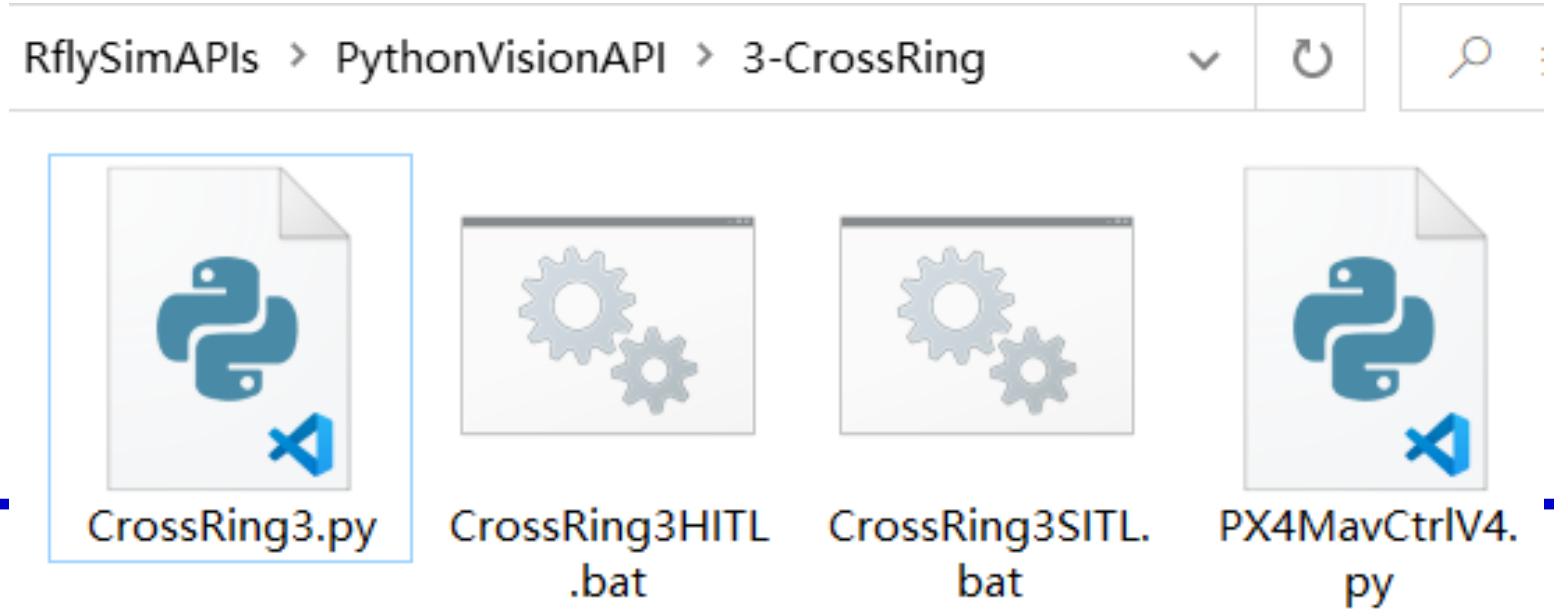
双击例程提供的**bat**脚本可以快速启动软/硬在环软件和期望数量的三维程序

ENG 11:49 PM 5/30/2020

3.单目视觉控制例子

3.4 无人机穿环实验

- 在Windows资源管理器中，打开并进入“RflySimAPIs\PythonVisionAPI\3-CrossRing”文件夹，其中的内容如下图
- 其中，“CrossRing3.py”是本例程的主Python程序；“ShootBall3HITL.bat”和“ShootBall3SITL.bat”相对于上一个撞击小球的例子的区别在于：“UE4_MAP”地图场景变量选择了用于视觉穿环场景“VisionRing”。注其“UDPSIMMODE”通信UDP模式也选择了“Mavlink_Full”模式。



3.单目视觉控制例

3.5 穿环实验代码解析

```python
def saturationYawRate(yaw_rate):
    yr_bound = 20.0
    if yaw_rate > yr_bound:
        yaw_rate = yr_bound
    if yaw_rate < -yr_bound:
        yaw_rate = -yr_bound
    return yaw_rate

def taskChange(pos_x):
    if pos_x < 40:
        task = "range1"
    elif pos_x <70:
        task = "range2"
    elif pos_x < 130:
        task = "range3"
    elif pos_x < 140:
        task = "land"
    else:
        task = "finish"
    return task
```

1.偏航饱和函数

2.位置触发的任务切换状态机函数

```python
def objectDetect(task):
    """According task to detect objects"""
    img_rgba=getCVImg(ImgInfo1)
    img_bgr = cv2.cvtColor(img_rgba, cv2.COLOR_RGBA2RGB)
    #img_bgr = cv2.cvtColor(img_rgba, cv2.COLOR_RGBA2RGB)
    img_bgr = cv2.resize(img_bgr, (width, height))

    #print("objectDetect")
    #cv2.imshow("img_bgr1", img_bgr)
    b,g,r = cv2.split(img_bgr)
    img_edge = cv2.Canny(b, 50, 100)
    #print("objectDetect1")
    if task == "range1" or task == "range2":
        #print("objectDetect2")
        return circleDetect(img_bgr, img_edge, b)
    else:
        return squareDetect(img_bgr, img_edge)
```

3.目标检测算法函数

```python
def squareDetect(img_bgr, img_edge):
    """Detect Square with PolyDP and diagonal length"""
    # find contours
    squares = []
    cnts, hierarchy = cv2.findContours(img_edge, cv2.RETR_LIST, cv2.CHAIN_APPROX_SIMPLE)
    # cv2.drawContours(img_bgr, cnts, -1, (0,255,0), 3)
    for cnt in cnts:
        cnt_len = cv2.arcLength(cnt, True)
        cnt = cv2.approxPolyDP(cnt, 0.05 * cnt_len, True)
        if len(cnt) == 4 and cv2.contourArea(cnt) > 2000 and cv2.isContourConvex(cnt):
```

4.方形检测函数

3.单目视觉控制例子

3.5 穿环实验代码解析

5.圆形检测函数

```python
def circleDetect(img_bgr, img_edge, img_b):
    """Hough Circle detect"""
    #print("objectDetect3")
    circles = cv2.HoughCircles(img_b, cv2.HOUGH_GRADIENT, 1, 20,
    #print("objectDetect4")
    if circles is not None:
        circles = np.uint16(np.around(circles))
        obj = circles[0, 0]
        cv2.circle(img_bgr, (obj[0], obj[1]), obj[2], (0,255,0),
        cv2.circle(img_bgr, (obj[0], obj[1]), 2, (0,255,255), 3)
        #cv2.imshow("img_edge", img_edge)
        cv2.imshow("img_bgr", img_bgr)
        cv2.waitKey(1)
        height, width, channel = img_bgr.shape
        return obj[0]-width/2, obj[1]-height/2, obj[2]
    else:
        return -1,-1,-1
```

6.接近目标控制函数

```python
def approachObjective():
    # 0. parameters
    # some parameters that work:(0.03, -0.03, 1, 5.0); (0.
    K_z = 0.004 * 640 / height
    K_yawrate = 0.005 * 480 / width
    task = "range1"
    # 1. start
    startAppTime= time.time()
    while (task != "finish") & (task != "land"):
        # 1.1. detect object and get error with center of
        ex, ey, r = objectDetect(task)
        # 1.2. where is the drone
        pos_x = mav.uavPosNED[0]
        #print(time.time())
        # 1.3. update task
        task = taskChange(pos_x)
        # 1.4. attack
        if ex != -1:
            vx = sat(time.time()-startAppTime,3)
            vy = 0
            vz = K_z * ey
            yawrate = K_yawrate * ex
            mav.SendVelFRD(vx, vy, vz, yawrate)
    while task == "land":
        pos_x = mav.uavPosNED[0]
        #print(pos_x)
        task = taskChange(pos_x)
        mav.SendVelFRD(0, 0, 1, 0)
```

3.单目视觉控制例子

3.5 穿环实验代码解析

```
230  if __name__ == '__main__':
231      mav = PX4MavCtrl.PX4MavCtrler(20100)
232      mav.InitMavLoop()
233      mav.sendUE4Cmd(b'RflyChangeMapbyName VisionRing',0)
234      mav.sendUE4Cmd(b'RflyChangeMapbyName VisionRing',1)
235      time.sleep(0.5)
236      mav.sendUE4Cmd(b'RflyChangeViewKeyCmd V 1',1)
237      time.sleep(0.5)
238      mav.sendUE4Cmd(b'RflyCameraPosAng 0.3 0 0 0 0 0',1)
239
240      window_hwnds = []
241      win32gui.EnumWindows(window_enumeration_handler, window_hwnds)
242
243
244      print(len(window_hwnds))
245      if len(window_hwnds)<2:
246          print("RflySim3D窗口数小于2个，本程序无法正常运行")
247          mav.stopRun()
248          sys.exit(1)
249
250      mav.sendUE4Cmd(b'r.setres 720x405w',1)
251      time.sleep(2)
252
```

主函数：初始化MAVLink，设置RflySim3D显示，获取图像窗口

```
280      print("Simulation Start.")
281      while True:
282          lastTime = lastTime + timeInterval
283          sleepTime = lastTime - time.time()
284          if sleepTime > 0:
285              time.sleep(sleepTime)
286          else:
287              lastTime = time.time()
288          #以下代码0.01s执行一次
289
290          if time.time() - startTime > 5 and flag == 0:
291              print("Enter Offboard mode.")
292              mav.initOffboard()
293              time.sleep(0.5)
294              mav.SendMavArm(True) #解锁命令
295              mav.SendPosNED(0, 0, -5, 0) #飞到目标点 0,0, -5位置，命令范围10s
296              flag = 1
297
298          if time.time() - startTime > 15 and flag == 1:
299              break
300      approachObjective()
301      # 结束，释放内存
```

0.01s的定时器

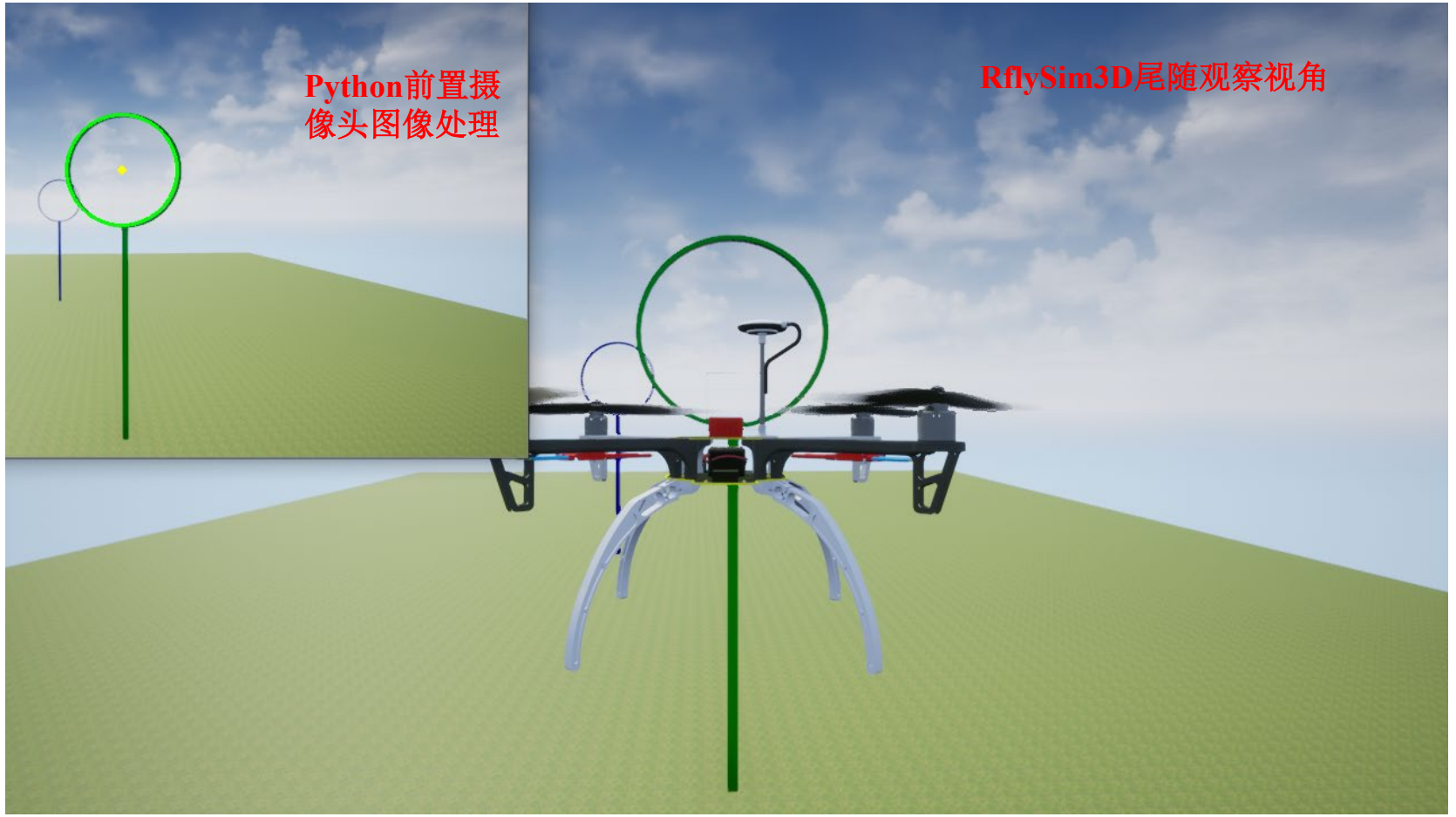
第5秒开始起飞

第15秒开始执行穿环检测并给CopterSim发送实时控制量

3.单目视觉控制例子

3.6 穿环实验运行效果

- 双击运行“CrossRing3SITL.bat”文件开启软件在环仿真系统，再运行“CrossRing3.py”程序。
- 飞机起飞后按照顺序穿过三个环，最后自动降落。
- 若要使用硬件在环仿真，设置好飞控后，运行“CrossRing3HITL.bat”脚本，并输入飞控串口号，来开启硬件在环仿真系统。

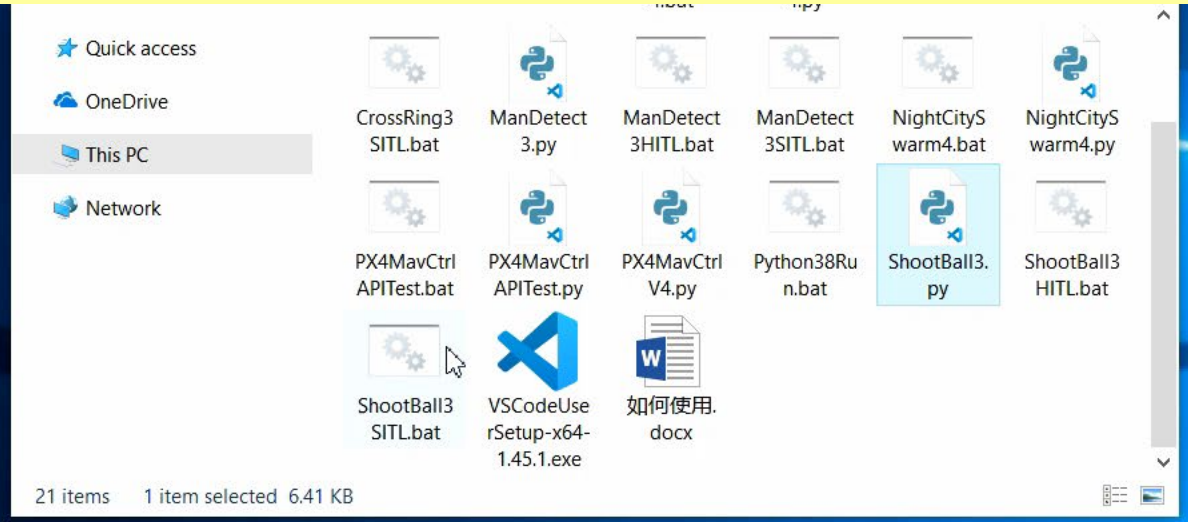


RflySim. 多旋翼无人机进行视觉导航控制 —— 多旋翼穿环实验

本视频观看地址：

优酷： https://v.youku.com/v_show/id_XNDcwNjA4NTYwNA==.html

YouTube： https://youtu.be/PvxEfY7oMq4



双击例程提供的**bat**脚本可以快速启动软/硬在环软件和期望数量的三维程序

ENG 11:49 PM 5/30/2020

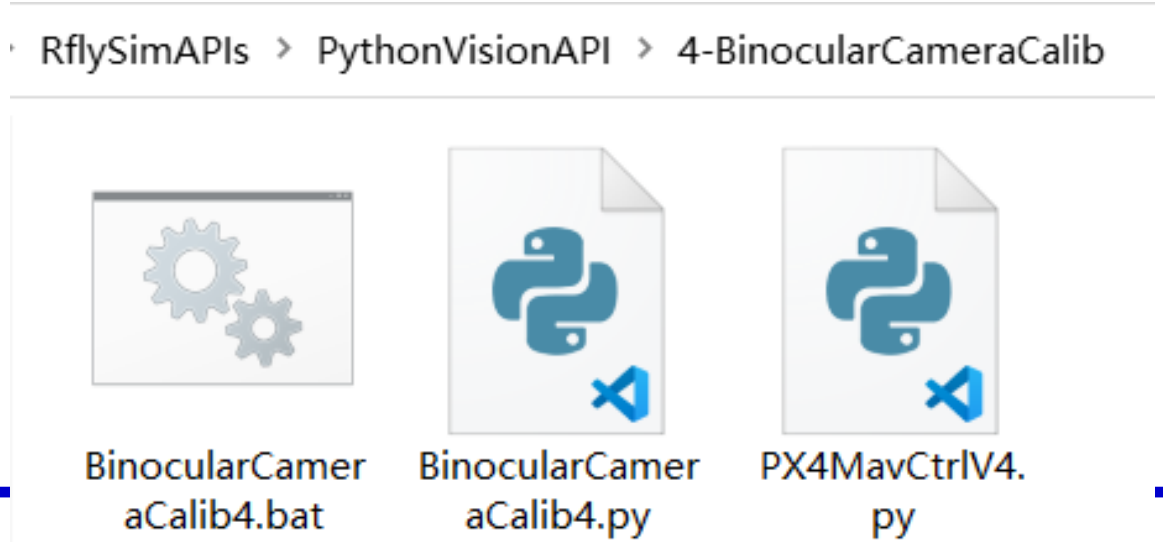
大纲

1. 安装使用方法
2. 基础接口使用
3. 单目视觉控制例子
4. 双目视觉控制例子
5. 小结

4.双目视觉控制例子

4.1 双目相机标定实验

- 在Windows资源管理器中，打开并进入“RflySimAPIs\PythonVisionAPI\4-BinocularCameraCalib”文件夹，其中的内容如下图
- 其中，“BinocularCameraCalib4.py”是本例程的主Python程序；“BinocularCameraCalib4.bat”是一个批处理启动脚本，双击它会自动打开三个RflySim3D窗口（左右两个摄像头+尾随全局观察视角）；“PX4MavCtrlV4.py”是本平台的Python通信接口模块。



4.双目视觉控制例子

4.2 双目相机标定代码解析

- 用VS Code打开“BinocularCameraCalib4.py”文件
- 关键代码行如右图所示，本脚本可以同时获取多个窗口的图像
- 其余代码请根据前边的讲解自行阅读学习

```
205  ImgInfo1 = getHwndInfo(Wd01)
206  ImgInfo2 = getHwndInfo(Wd02)
207
208
209  startTime = time.time()
210  lastTime = time.time()
211  timeInterval = 3
212  #face_cascade=cv2.CascadeClassifier('.\cascades\haarcascade_frontalface_default.xml')
213  while True:
214      lastTime = lastTime + timeInterval
215      sleepTime = lastTime - time.time()
216      if sleepTime > 0:
217          time.sleep(sleepTime)
218      else:
219          lastTime = time.time()
220      TargePos = [InitTargePos[0]+random.randint(0,100)/100.0*2,InitTargePos[1]+random.randint(-50,50)/50.0,InitT
221      TargeAng = [InitTargeAng[0]+random.randint(-50,50)/50.0*30/180.0*math.pi,InitTargeAng[1]+random.randint(-50
222      # print(TargePos)
223      # print(TargeAng)
224      mav.sendUE4Pos(100,40,0,TargePos,TargeAng,-1)
225      time.sleep(0.1)
226
227      #以下代码0.01s执行一次
228      img1=getCVImg(ImgInfo1)
229      #转为灰度图
230      pic1=cv2.cvtColor(img1, cv2.COLOR_BGRA2GRAY)
231      img2=getCVImg(ImgInfo2)
232      #转为灰度图
233      pic2=cv2.cvtColor(img2, cv2.COLOR_BGRA2GRAY)
```

获取左右两个窗口的图像信息

时间间隔3s

定时器设置3s一个循环

随机设置靶标的位置和姿态

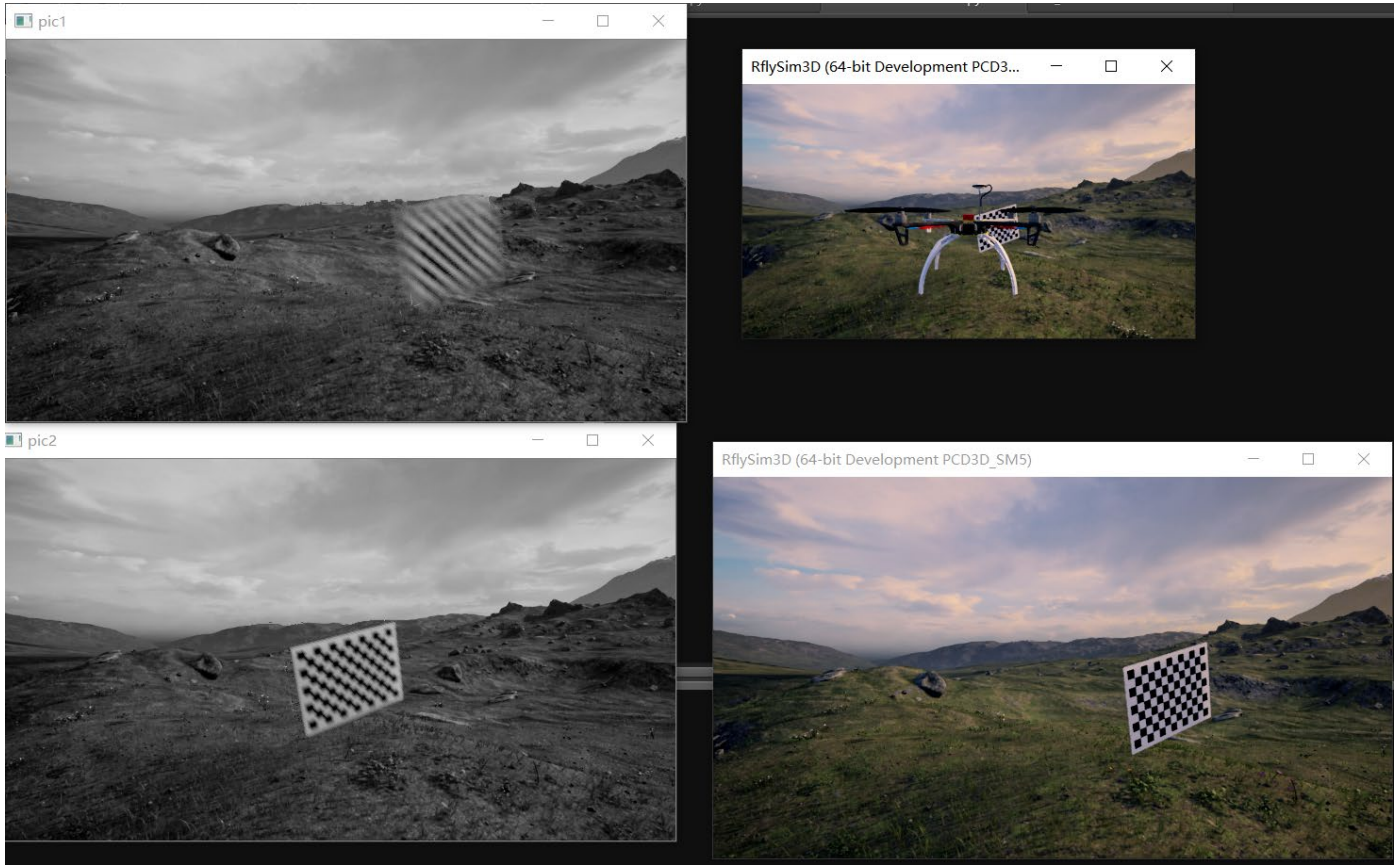
获取左侧相机的图像

获取右侧相机图像

4.双目视觉控制例子

4.3 双目相机标定实验效果

- 运行“BinocularCameraCalib4.bat”后，再运行“BinocularCameraCalib4.py”即可。
- 开启多个RflySim3D场景，新建一个飞机，配置双目位置信息，新建一个靶标，让靶标按随机规则摆放。
- 作业1：在获取到左右两个相机的图像后，实现在线的标定算法。
- 作业2：将左右两个相机的图像以图片形式存储到本地，然后用MATLAB的标定工具箱求参数



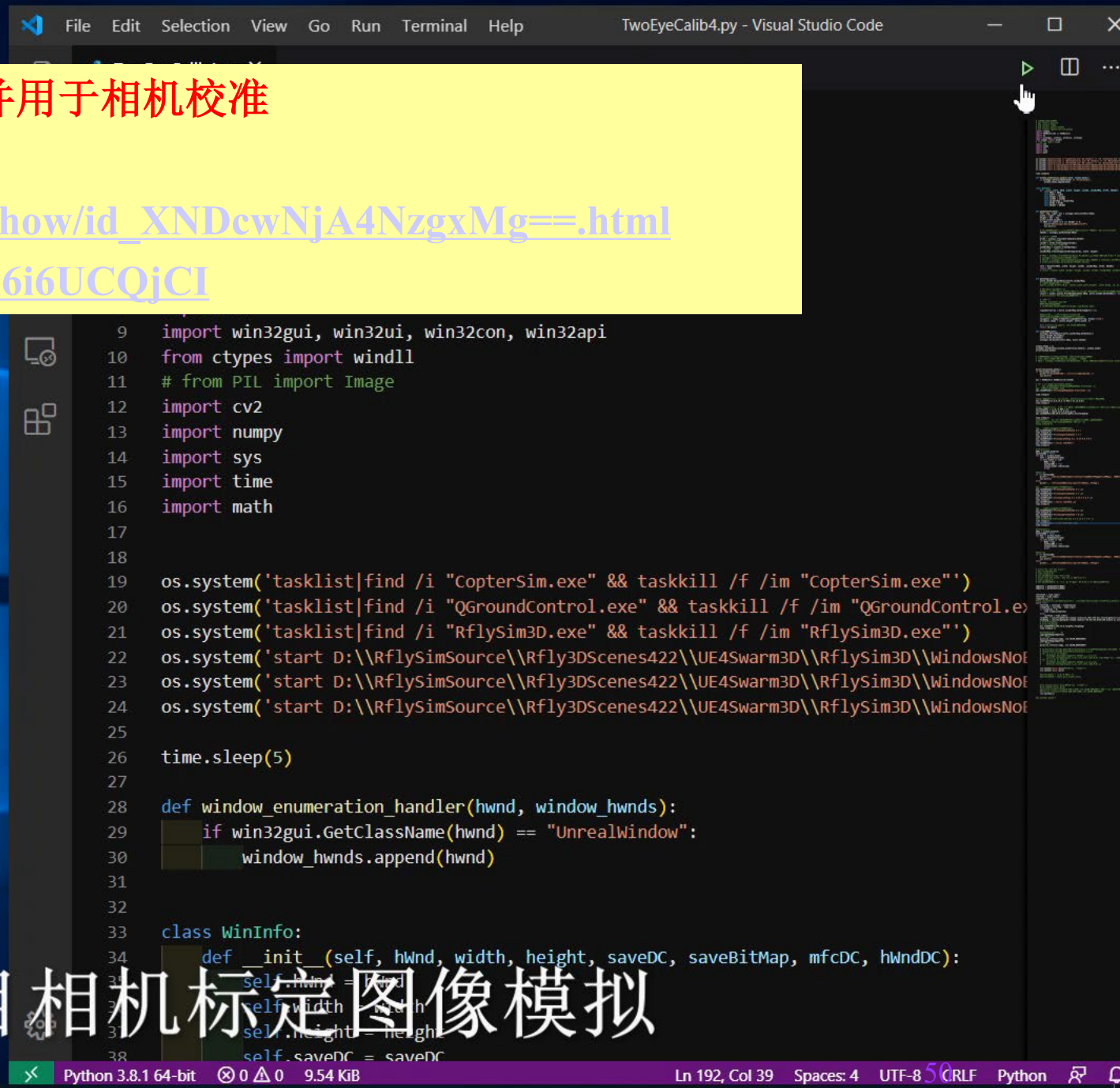
RflySim：获取双目相机图像并用于相机校准

本视频观看地址：

优酷： https://v.youku.com/v_show/id_XNDcwNjA4NzgxMg==.html

YouTube： https://youtu.be/hm6i6UCQjCI

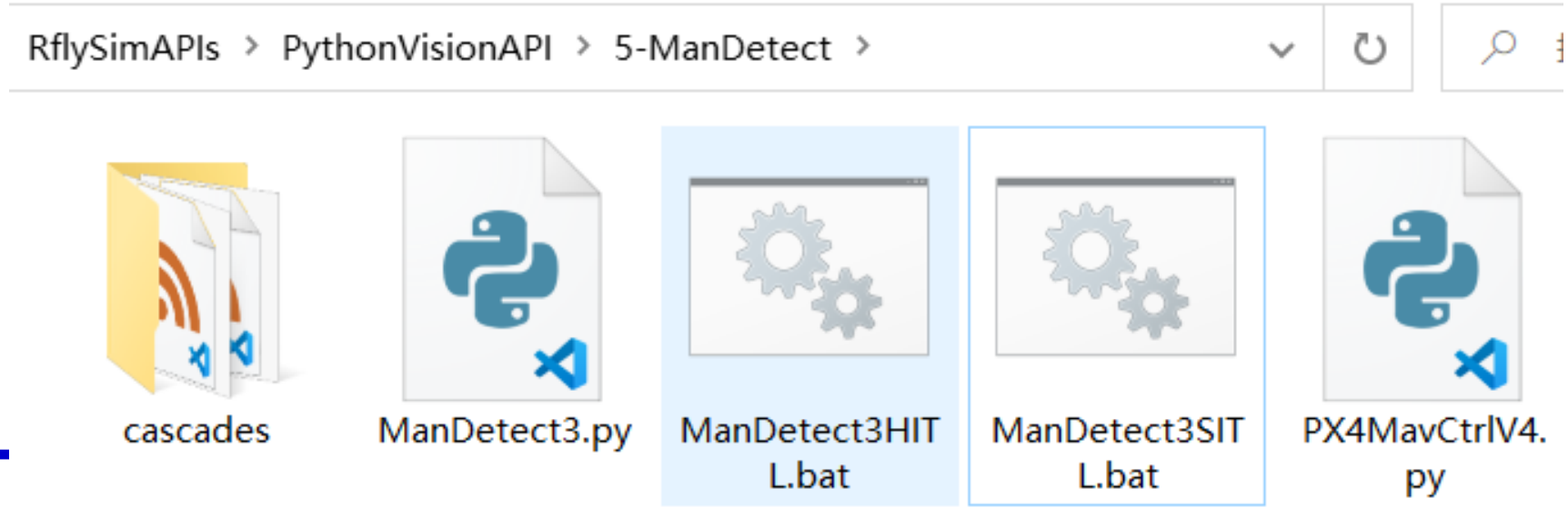
自动创建双目相机标定图像模拟



4.双目视觉控制例子

4.4 无人机双目视觉例程

- 在Windows资源管理器中，打开并进入“RflySimAPIs\PythonVisionAPI\5-ManDetect”文件夹，其中的内容如下图
- 其中，“ManDetect3.py”是本例程的主Python程序；cascades文件夹内装了人脸识别的一些XML格式的特征文件；“ManDetect3HITL.bat”和“ManDetect3SITL.bat”相对于桌面快捷方式的区别在于：“UDPSIMMODE”通信UDP模式也选择了“Mavlink_Full”模式；开启了三个RflySim3D窗口，左右相机+尾随观察视角。



4.双目视觉控制例子

4.5 无人机双目视觉源码解析

- 用VS Code打开“ManDetect3.py”文件

```
113  mav = PX4MavCtrl.PX4MavCtrler(20100)
114  mav.InitMavLoop()
115  # for i in range(len(window_hwnds)):
116  #     mav.sendUE4Cmd(b'RflyChangeMapbyName Grasslands',i)
117  #向所有窗口发送改变地图命令
118  mav.sendUE4Cmd(b'RflyChangeMapbyName Grasslands')
119
120  time.sleep(5)
121
122  #发送位置指令，创建一个人物，设置ID为100，三维中飞本程中的x,y位置在飞机前面
123  mav.sendUE4Pos(100,30,0,[1,0,-8.086],[0,0,math.pi])
124
125  time.sleep(1)
126  #发送消息，使所有场景中的ID100（刚创建的人物）变成行走的人的样式
127  mav.sendUE4Cmd(b'RflyChange3DModel 100 16')
128  time.sleep(0.5)
129
130  #向第一个窗口发送指令，改变窗口尺寸
131  mav.sendUE4Cmd(b'RflyChangeViewKeyCmd B 1',0)
132  time.sleep(0.5)
133  mav.sendUE4Cmd(b'RflyChangeViewKeyCmd V 1',0)
134  time.sleep(0.5)
135  mav.sendUE4Cmd(b'RflyCameraPosAng 0.1 -0.25 0',0)
136  time.sleep(1)
137  mav.sendUE4Cmd(b'r.setres 720x405w',0)
138  time.sleep(2)
```

初始化MAVLink通信并设置RflySim3D的左右摄像头位置

```
221  #下面这里是图片处理的主代码
222  startTime = time.time()
223  lastTime = time.time()
224  timeInterval = 0.01
225  face_cascade=cv2.CascadeClassifier(sys.path[0]+'\cascades\haarcascade_frontal
226  while True:
227      lastTime = lastTime + timeInterval
228      sleepTime = lastTime - time.time()
229      if sleepTime > 0:
230          time.sleep(sleepTime)
231      else:
232          lastTime = time.time()
233      #以下代码0.01s执行一次
234      img1=getCVImg(ImgInfo1)
235      #转为灰度图
236      pic1=cv2.cvtColor(img1, cv2.COLOR_BGRA2GRAY)
237      img2=getCVImg(ImgInfo2)
238      #转为灰度图
239      pic2=cv2.cvtColor(img2, cv2.COLOR_BGRA2GRAY)
240
241      faces1=face_cascade.detectMultiScale(pic1,1.3,5)#分别对左右两部分做人脸检测
242      faces2=face_cascade.detectMultiScale(pic2,1.3,5)
243      for (x,y,w,h) in faces1:
244          #win32gui.Rectangle(ImgInfo1.hWndDC,x,y,x+w,y+h)
245          pic1=cv2.rectangle(pic1,(x,y),(x+w,y+h),(255,0,0),1)#在灰度图上画出人脸
246      for (x,y,w,h) in faces2:
247          #win32gui.Rectangle(ImgInfo2.hWndDC,x,y,x+w,y+h)
248          pic2=cv2.rectangle(pic2,(x,y),(x+w,y+h),(255,0,0),1)
249
250      cv2.imshow("pic1",pic1)#分别显示检测后的图像
251      cv2.imshow("pic2",pic2)
252
```

每0.01s一个循环

加载人脸特征文件

定时器

获取双目图像

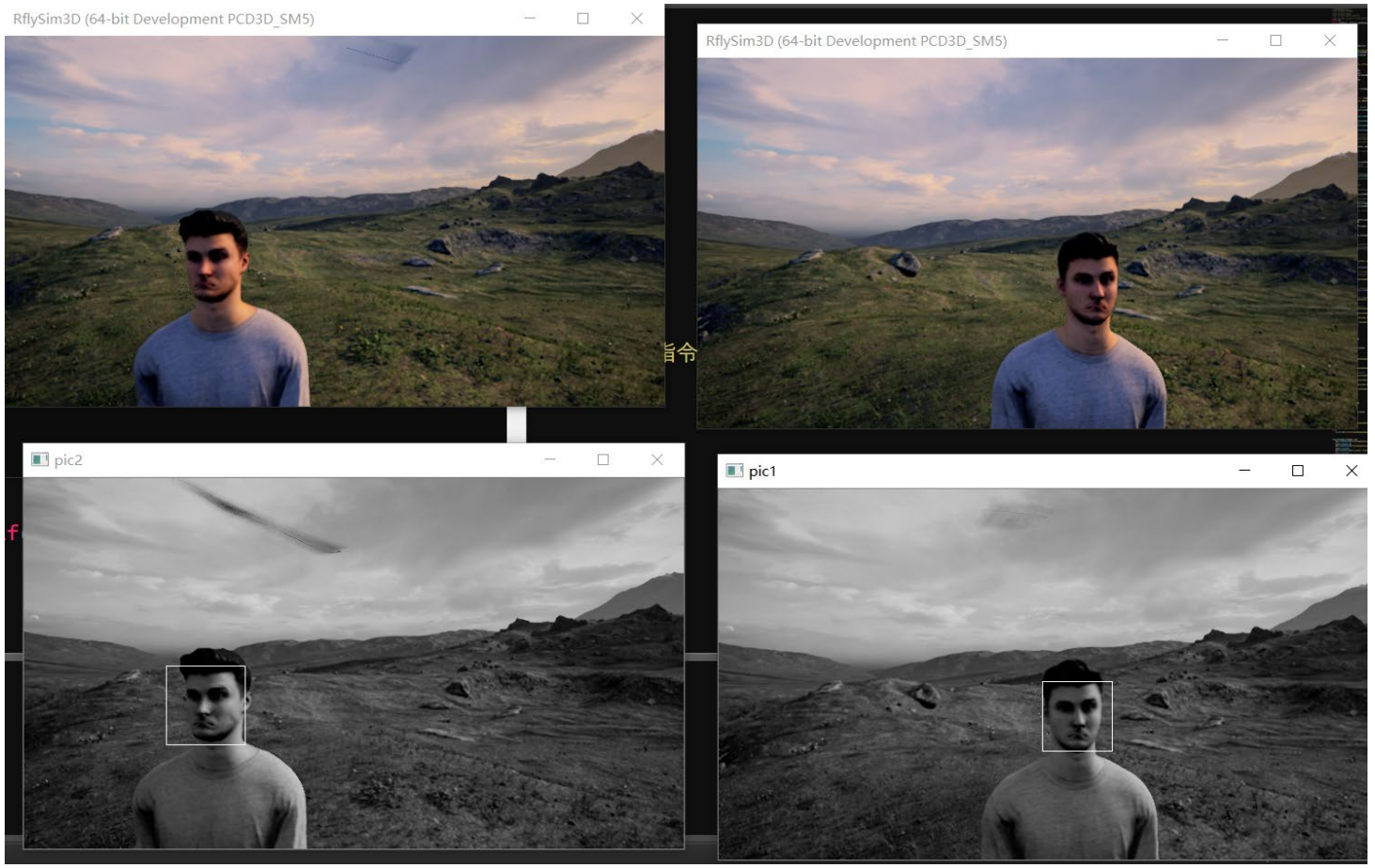
调用OpenCV目标检测

显示检测结果

4.双目视觉控制例子

4.6 无人机双目视觉运行效果

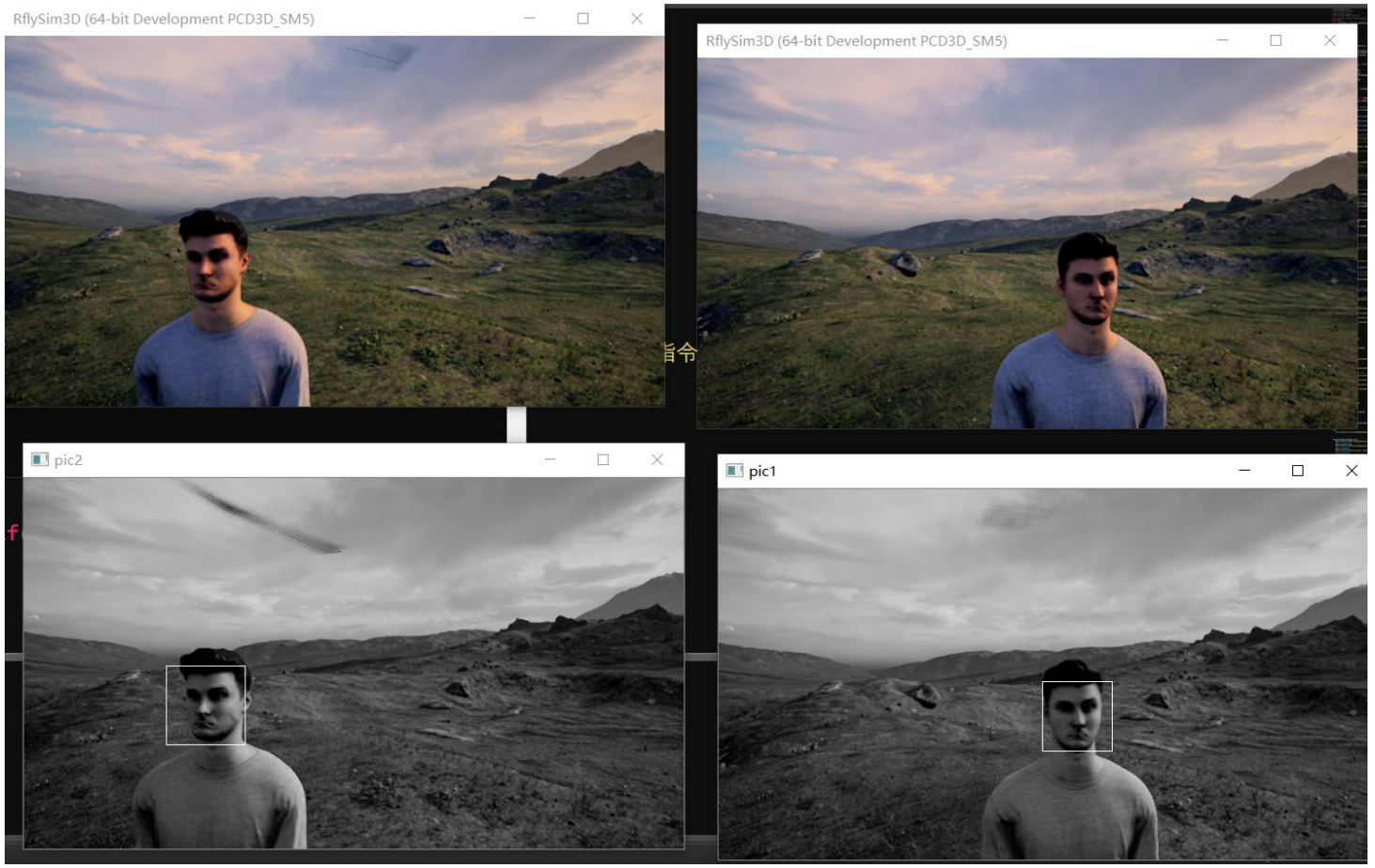
- 运行“ManDetect3SITL.bat”或“ManDetect3HITL.bat”，再运行“ManDetect3.py”即可
- RflySim3D中生成走动的人，并设置面对飞机，飞机起飞后开启人脸识别算法，双目框选出人脸
- 作业1：实时更新人的位置，实现人物走动的模拟，并编写飞机跟踪控制器
- 作业2：改成前视+下视摄像头，验证跟踪+光流算法。



4.双目视觉控制例子

4.6 无人机双目视觉运行效果

- 运行“ManDetect3SITL.bat”或“ManDetect3HITL.bat”，再运行“ManDetect3.py”即可
- RflySim3D中生成走动的人，并设置面对飞机，飞机起飞后开启人脸识别算法，双目框选出人脸
- 作业1：实时更新人的位置，实现人物走动的模拟，并编写飞机跟踪控制器
- 作业2：改成前视+下视摄像头，验证跟踪+光流算法。



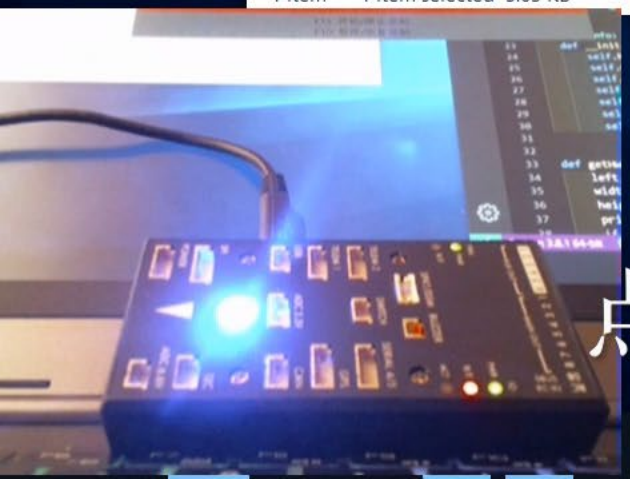
RflySim：获取双目相机图像并进行人脸识别

本视频观看地址：

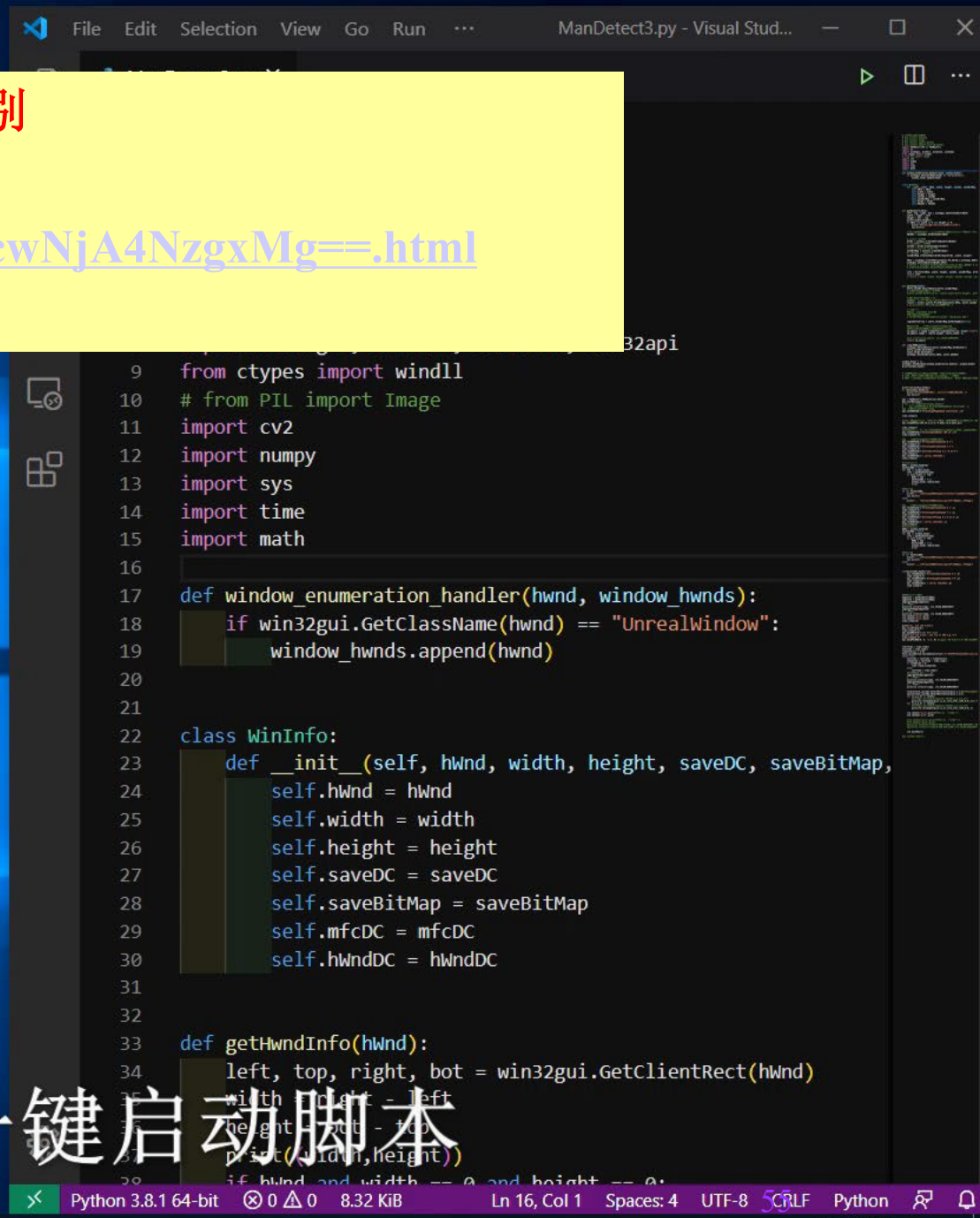
优酷： https://v.youku.com/v_show/id_XNDcwNjA4NzgxMg==.html

YouTube： https://youtu.be/hm6i6UCQjCI





点击HIL硬件在环一键启动脚本



51CTO

4.双目视觉控制例子

4.7 无人机视觉算法的部署

- 将视觉控制的Python代码拷贝到机载电脑上，将RflySim3D的视觉图像换成摄像机图像，即可完成视觉算法的部署







RflySim：双目飞行平台介绍

本视频观看地址：

优酷： https://v.youku.com/v_show/id_XNDcwNjA4NzgxMg==.html

YouTube： https://youtu.be/hm6i6UCQjCI

Vision&Sw



ance 10min

- Onboard communication interface for vision&Swarm
- Optical flow & laser ranger finder for indoor experiments
- M8N GPS for outdoor experiments
- Visible light camera & Binocular camera
- Onboard TX2 or Raspberry Pi system
- Labels for Optitrack and UWB system

谢谢！